



**The Hashemite University, Zarqa, Jordan
Faculty of Prince Al-Hussein Bin Abdallah II for Information Technology
Software Engineering Department**

CVision

**A project submitted
in partial fulfillment of the requirements for the
B.Sc. Degree in Software Engineering**

By

Hiba Emad Alhjaj (2238493)
Dareen Marwan Barakat (2237254)
Noor Harbi Alnatsheh (2231823)
Reham Tariq Alhasabeen (2233299)

Supervised by

Dr.Khaled Abdelkarim Almakadmeh

Committee Member Names

Dr.Hani Ahmad Bani-Salameh

January 2026

CERTIFICATE

It is hereby certified that the project titled <*CVision*>, submitted by undersigned, in partial fulfillment of the award of the degree of “Bachelor’s in Software Engineering” embodies original work done by them under my supervision.

All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

Hiba Alhjjaj (2238493)



Dareen Barakat (2237254)



Noor Alnatsheh (2231823)



Reham Alhasabeen (2233299)



ABSTRACT

CVision is an intelligent tool that analyzes team members' Curriculum Vitae (CVs) to support role assignment and project planning in software development teams. It addresses the common challenge of matching roles to skills while reducing subjective and opinion-based decisions.

Using Artificial Intelligence (AI), CVision analyzes uploaded PDF or Word-format CVs and extracts relevant information, including technical skills, soft skills, tools, certifications, and years of experience. The system also evaluates extracted skills and assigns proficiency scores based on the evidence found within the CV.

The platform provides individual and team dashboards that summarize members' strengths and competencies, helping project managers understand the overall skill distribution within a team.

A role matching engine compares project role requirements with team members' skill profiles. The system calculates match scores using both skill proficiency levels and role requirement priorities, then ranks candidates according to their suitability for each role. Project managers can review recommendations and manually override assignments when needed.

CVision also identifies skill gaps within the team and highlights missing competencies required for project roles. Based on these gaps, the system can recommend potential development opportunities, such as training existing members or adding new team members with the required expertise.

The expected impact of CVision is improved role assignment accuracy, better utilization of team capabilities, enhanced project planning, and increased visibility into team strengths and weaknesses, enabling more informed management decisions.

ACKNOWLEDGEMENTS

Special thanks to our supervisor We extend our deepest gratitude to Dr Khaled Al-makadmeh, our project supervisor, whose guidance, expertise, and patience were indispensable throughout this graduation project. Your clear direction, constructive feedback, and constant encouragement helped us refine our ideas and deliver this work to a high standard.

TABLE OF CONTENTS

CERTIFICATE	II
ABSTRACT.....	III
ACKNOWLEDGEMENTS	IV
TABLE OF CONTENTS.....	V
ABBREVIATIONS	VII
LIST OF FIGURES	VIII
LIST OF TABLES.....	X
Chapter 1: Introduction	1
1.1 Overview	1
1.2 Project Motivation.....	1
1.3 Problem Statement	1
1.4 Project Aim and Objectives.....	2
1.5 Project Scope.....	2
1.6 Project Limitations	5
1.7 Project Expected Output.....	5
1.8 Project Schedule	5
1.9 Project, product, and schedule risks	6
1.10 Report Organization	7
Chapter 2: Literature Review	8
2.1 Introduction	8
2.2 Existing Systems	8
2.3 Overall Problems of Existing Systems.....	9
2.4 Overall Solution Approach.....	10
Chapter 3: Requirement Engineering and Analysis	11
3.1 Stakeholders	11

3.2	Use Case Diagram	12
3.3	Non-Functional User Requirements	29
3.3	Constraints	31
Chapter 4: Architecture and Design		32
4.1	Overview	32
4.2	Software Architecture	32
4.3	Software design	36
4.4	User interface design (prototype)	45
Chapter 5: Implementation Plan		52
5.1	Description of Implementation	52
5.2	Programming language and technology	53
5.3	Version Control	54
Chapter 6: Testing Plan		55
6.1	Black-box	56
6.2	White-box	58
6.3	Testing automation	59
Chapter 7: Conclusion and Results		60
7.1	Summary of accomplished project	60
7.2	Future Work	61
References		62

ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CV	Curriculum Vitae
CVision	Smart CV Analysis and Role Assignment System
DB	Database
GUI	Graphical User Interface
ORM	Object Relational Mapping
JSON	JavaScript Object Notation
SQL	Structured Query Language
LLM	Large Language Model
REST	Representational State Transfer
PDF	Portable Document Format
UI	User Interface
UX	User Experience

LIST OF FIGURES

Figure 1: “Use Case Diagram”	12
Figure 2: “Logical View Diagram”	32
Figure 3: “Process Diagram”	33
Figure 4: “Physical Diagram”	33
Figure 5: “Sign up as a team member”	36
Figure 6: “Sign up as a project manager”	37
Figure 7: “Enter project requirment”	37
Figure 8: “Create project”	38
Figure 9: “Join project”	38
Figure 10: “Sign in”	39
Figure 11: “Upload CV & View assigned role”	39
Figure 12: “Enter project requirment & role”	40
Figure 13: “Class Diagram”	41
Figure 14: “ER Diagram”	42
Figure 15: “View team dashboard process”	43
Figure 16: “Upload CV process”	44
Figure 17: “Log in”	45
Figure 18: “Sign up as a team member”	45
Figure 19: “My project”	46
Figure 20: “Join project”	46
Figure 21: “Upload CV”	46
Figure 22: “CV analysis complete”	47
Figure 23: “My Skills”	47
Figure 24: “Sign up as a project manager”	47

LIST OF FIGURES

<u>Figure 25: “My project-PM”</u>	48
<u>Figure 26: “Create new project”</u>	48
<u>Figure 27: “Project details”</u>	48
<u>Figure 28: “Add new role”</u>	49
<u>Figure 29: “Edit role”</u>	49
<u>Figure 30: “Skills profile for member”</u>	49
<u>Figure 31: “Team members”</u>	50
<u>Figure 32: “Role matching & assignment”</u>	50
<u>Figure 33: “Skills gap analysis”</u>	50
<u>Figure 34: “Training recommendations”</u>	51

LIST OF TABLES

Table 1: “Abbreviations table”	VII
Table 2: “Project Schedule”	6
Table 3: “Comparison table”	9
Table 4: “Sign up as a project manager”	13
Table 5: “Sign up as a team member”	14
Table 6: “Verify account”	15
Table 7: “Sign in”	16
Table 8: “Forget password”	17
Table 9: “Create project”	18
Table 10: “Enter project requirements & roles”	19
Table 11: “View team dashboard”	20
Table 12: “View team skills”	21
Table 13: “Project manager”	22
Table 14: “Review missing skills / Roles”	23
Table 15: “Join Project”	24
Table 16: “Enter Project Link”	25
Table 17: “Upload CV”	26
Table 18: “View Individual Dashboard”	27
Table 19: “View Assigned Role”	28
Table 20: “Test Case”	57

CHAPTER 1: INTRODUCTION

1.1 Overview

CVision is an AI-powered software tool that analyzes team members' CVs to support role assignment and project planning in software development teams. The system extracts and evaluates skills, experience, tools, and certifications from uploaded CVs, then matches team members to project roles based on their qualifications and project requirements. CVision also helps identify skill gaps and provides insights that support better team management and decision-making.

1.2 Project Motivation

We chose to develop CVision because software development teams often face difficulties in assigning the right people to the right roles. Traditional assignment methods are frequently based on personal impressions, limited knowledge of team members' capabilities, or manual review of CVs, which can be time-consuming and lead to inaccurate decisions.

The project aims to improve the role assignment process by providing a structured and data-driven approach for evaluating team members' qualifications. Using Google Gemini AI, CVision analyzes uploaded PDF and Word-format CVs, extracts relevant information such as technical skills, soft skills, tools, certifications, and years of experience, and evaluates skill proficiency based on the evidence available in the CV.

The system then compares these skills with project role requirements and calculates matching scores using both skill proficiency and role priorities defined by the Project Manager. Furthermore, CVision identifies missing skills within the team and provides recommendations that support better planning, team development, and resource allocation.

By transforming unstructured CV information into meaningful insights, CVision helps reduce assignment errors, improve team utilization, increase transparency in decision-making, and support more effective project management within software development teams.

1.3 Problem Statement

In software development teams, assigning the right roles to the right people is often performed manually and based on personal judgment or limited knowledge of team members' actual capabilities. This approach can lead to skill mismatches, inefficient

resource utilization, and reduced team productivity. Traditional methods, such as manual CV review or informal discussions, are time-consuming, subjective, and may not accurately reflect the full range of technical and soft skills possessed by team members.

To address these challenges, CVision provides an AI-powered, data-driven solution that automatically analyzes team members' CVs, extracts relevant skills, experience, tools, and certifications, and evaluates their proficiency levels. The system then compares individual competencies with project requirements to recommend suitable role assignments and identify skill gaps within the team. By providing objective and structured insights, CVision helps reduce bias, improve role assignment accuracy, and support more effective project planning and team management.

1.4 Project Aim and Objectives

Project Aim:

The main aim of this project is to develop an AI-powered software tool, CVision, that automatically analyzes team members' CVs to support accurate role assignment, effective project planning, and better utilization of team skills within software development teams.

Project Objectives:

To achieve this aim, the project pursues the following objectives:

- **Skill Extraction and Evaluation:**
Automatically extract and evaluate technical skills, soft skills, tools, certifications, and experience from uploaded CVs using Artificial Intelligence.
- **Skill Proficiency Assessment:**
Assess the proficiency level of extracted skills and generate skill profiles that reflect each team member's strengths and competencies.
- **Smart Role Matching:**
Recommend suitable team members for project roles by comparing role requirements with skill profiles and calculating matching scores based on skill proficiency and role priorities.
- **Skill Gap Analysis:**
Identify missing skills and competency gaps within the team by comparing available capabilities against project requirements.

- **Performance Visualization:**

Provide interactive dashboards that summarize individual and team strengths, skill distributions, role matching results, and skill gap insights to support better managerial decision-making.

- **Improved Team Planning:**

Support Project Managers in making more informed, objective, and data-driven decisions regarding team composition, role assignment, and resource allocation.

1.5 Project Scope

This project focuses on developing an AI-powered tool, CVision, that analyzes team members' CVs to support role assignment and project planning. The system includes CV analysis, skill extraction and evaluation, role matching, skill gap identification, and dashboard visualization to help Project Managers make informed team management decisions.

Included Features:

- Automatic CV analysis and skill extraction using AI.
- Skill scoring to quantify each member's proficiency.
- Skill gap identification within the team.
- Interactive dashboards for individual and team skill insights.
- Smart role matching based on project requirements and skill priorities.

Excluded Features:

- Full HR recruitment automation.
- Real-time communication or chat features.
- LinkedIn/GitHub integration.
- Project tracking and task management features.

Team Responsibilities:

The project work is distributed among team members according to their assigned responsibilities, including backend development, AI integration, frontend implementation, UI/UX design, database management, and system integration.

Documentation, testing, and presentation activities are carried out collaboratively by the entire team.

Verification and Approval:

Work will be verified through regular supervisor reviews, internal team testing, and peer evaluation after each project milestone before final submission.

Internal testing refers to testing performed by the project team before presenting the system to the supervisor. The team will verify that all core functionalities operate correctly, including CV uploading, AI-based CV analysis, skill extraction, dashboard visualization, role matching, and skill gap analysis. This process helps identify and resolve issues early, ensuring that the system functions as expected before formal evaluation and approval.

Project Software and Hardware Requirements:

Software Requirements:

- Programming Languages: Python (backend and AI integration), JavaScript (frontend).
- Frameworks and Tools: FastAPI, React, Figma (UI/UX Design), Draw.io (Diagrams).
- Databases: PostgreSQL (Neon Cloud Database).
- AI Services Google Gemini API for CV analysis and skill extraction.
- Development Tools: Visual Studio Code, Git, GitHub.
- Project Management: Trello.

Hardware Requirements:

- Development Machine: Minimum 8 GB RAM, dual-core or higher processor, and stable internet connection.
- Cloud Database: PostgreSQL database hosted on Neon.
- Storage: Minimum 10 GB for storing CV files and system data.

1.6 Project Limitations

- Skill extraction accuracy depends on the quality and completeness of uploaded CV.
- The system currently supports English-language CVs in PDF or Word formats.
- AI-generated results depend on the information available in the CV.
- Incomplete or outdated CV data may affect role matching accuracy.
- The system is intended for software development teams and not for full-scale recruitment management.

1.7 Project Expected Output

The final system will produce:

- **Analyzed CV data** containing extracted technical skills, soft skills, tools, certifications, and experience information, along with proficiency scores for each extracted skill.
- **Interactive dashboards** displaying individual and team skill profiles and competency distributions.
- **Role matching results** that compare project requirements with team members' qualifications and rank candidates based on their suitability.
- **Skill gap analysis** highlighting missing competencies required for project roles.

The expected outcome is improved role assignment accuracy, better utilization of team capabilities, enhanced project planning, and more informed decision-making within software development teams.

1.8 Project Schedule

Phase	Activity	Duration	Deliverables
Week 1–2	Requirement Gathering & Literature Review	2 weeks	Requirements Document
Week 3–4	System Design (Architecture, UML, UI Prototype)	2 weeks	Design Document, Diagrams
Week 5–6	Backend Development, Database Setup, and AI Integration	2 weeks	Functional Backend Prototype
Week 7–8	Frontend Development, Dashboard Implementation, and System Integration	2 weeks	Interactive User Interface
Week 9	Testing & Debugging	1 week	Test Report
Week 10	Final Documentation & Presentation	1 week	Final Report & Demo

1.9 Project, product, and schedule risks

- **Technical Risks** AI analysis accuracy may be affected by poorly formatted or incomplete CVs. External AI services may also experience temporary availability or quota issues.

Mitigation: Test the system using different CV formats, validate extracted data, and implement error handling for AI service failures.

- **Product Risks:** Role matching accuracy depends on the quality and completeness of the uploaded CV information.

Mitigation: Encourage users to upload updated CVs and allow Project Managers to manually review and override recommendations.

- **Schedule Risks:** Project delays may occur due to academic workload or unexpected technical challenges.

Mitigation: Divide tasks among team members, follow a weekly schedule, and regularly monitor progress with the supervisor.

- **Risk Mitigation Plan:** These risks will be managed through continuous testing, regular progress tracking, task distribution among team members, and ongoing communication with the supervisor.

1.10 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2** presents a literature review of existing tools and highlights gaps addressed by CVision.
- **Chapter 3** defines the requirement engineering process, including stakeholders, use cases, and non-functional requirements.
- **Chapter 4** details the system architecture, UML diagrams, and user-interface design.
- **Chapter 5** outlines the implementation plan, technologies used, and version-control strategy.
- **Chapter 6** describes the testing plan, including black-box, white-box, and automation testing.
- **Chapter 7** concludes the project, summarizes the accomplishments, and proposes future enhancements.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Role assignment in software development teams remains a significant challenge in project management. Traditional approaches often rely on manual CV reviews and personal judgment, which can lead to inaccurate role assignments and inefficient use of team skills. With the advancement of Artificial Intelligence (AI), automated CV analysis has become an effective way to evaluate team members' qualifications and support data-driven decision-making.

This chapter reviews existing systems related to CV analysis, skill assessment, and role recommendation. It highlights their strengths and limitations and explains how CVision addresses these challenges through AI-powered CV analysis, skill evaluation, role matching, and skill gap identification.

2.2 Existing Systems

Several systems and tools have been developed to support CV analysis, skill assessment, and workforce management. For example, LinkedIn Talent Insights provides workforce analytics and identifies trending skills, but it focuses on labor market trends rather than individual team analysis. Rezi and Resume.io help users improve their CVs through keyword optimization; however, they do not support team role assignment or project planning. The O*NET Database offers standardized information about occupations and skills, but it does not analyze uploaded CVs or generate personalized recommendations. Other skill analysis tools can identify technical competencies from resumes, yet they often lack role matching, skill gap analysis, and team-focused dashboards. In addition, some academic team formation systems help distribute members based on skills but typically rely on manually entered information rather than automated CV analysis.

System / Tool	Main Function	Strengths	Weaknesses
LinkedIn Talent Insights	Workforce analytics and skill trends	Access to large professional datasets; identifies trending skills	Lacks project-level analysis and role assignment capabilities
Rezi / Resume.io	Resume optimization and keyword suggestions	Improves CV quality and ATS compatibility	Focuses on individual resumes rather than team management
O*NET Database	Skills and occupation classification	Provides standardized skill and role information	Cannot analyze uploaded CVs or generate personalized recommendations
SkillRadar / Text Mining Tools	Automated skill identification from resumes	Extracts skills efficiently from CV content	Limited role matching, team analysis, and dashboard support

2.3 Overall Problems of Existing Systems

While existing tools provide valuable insights, several common issues persist:

1. **Limited Automation:** Many systems depend on manually entered information rather than automatically analyzing uploaded CVs.
2. **Lack of Project-Oriented Matching:** Most tools evaluate skills without directly relating them to specific project roles and requirements.
3. **Limited Team Insights:** Existing systems often focus on individuals and provide little visibility into overall team strengths, weaknesses, and skill gaps.
4. **Weak Decision Support:** Few systems assist Project Managers by recommending suitable candidates for roles based on skill profiles.
5. **Insufficient Skill Gap Analysis:** Most tools do not effectively identify missing competencies required for project success.

These limitations demonstrate the need for a unified system that combines AI-based CV analysis, skill evaluation, role matching, and team-level insights to support more effective project management decisions.

2.4 Overall Solution Approach

CVision addresses the limitations of existing systems by combining AI-powered CV analysis, skill evaluation, role matching, and interactive dashboards within a single platform. Unlike traditional solutions that focus only on individual CV assessment, CVision links project requirements with team members' actual capabilities.

The system automatically:

- Extracts technical skills, soft skills, tools, certifications, and experience from uploaded CVs.
- Evaluates skill proficiency levels based on the information available in the CV.
- Matches team members to project roles using skill profiles and role priorities.
- Identifies skill gaps by comparing project requirements with available team competencies.

By combining automated analysis with Project Manager oversight, CVision supports more objective, accurate, and efficient role assignment and project planning decisions.

CHAPTER 3: REQUIREMENT ENGINEERING AND ANALYSIS

3.1 Stakeholders

Stakeholders are individuals or groups who are affected by the development and use of CVision or have an interest in its outcomes.

Primary Stakeholders

Project Manager:

The Project Manager is the primary user of CVision. They use the system to analyze team skills, view role matching results, identify skill gaps, and make informed role assignment decisions.

Team Members:

Team members upload their CVs and receive skill analysis results. Their CV data serves as the main input for skill extraction, evaluation, and role matching.

Secondary Stakeholders

Academic Supervisors:

Supervisors evaluate the project and ensure that the system meets academic requirements and project objectives.

Software Development Teams:

Development teams benefit from improved role assignment, better utilization of skills, and increased visibility into team strengths and weaknesses.

3.2 Use Case Diagram

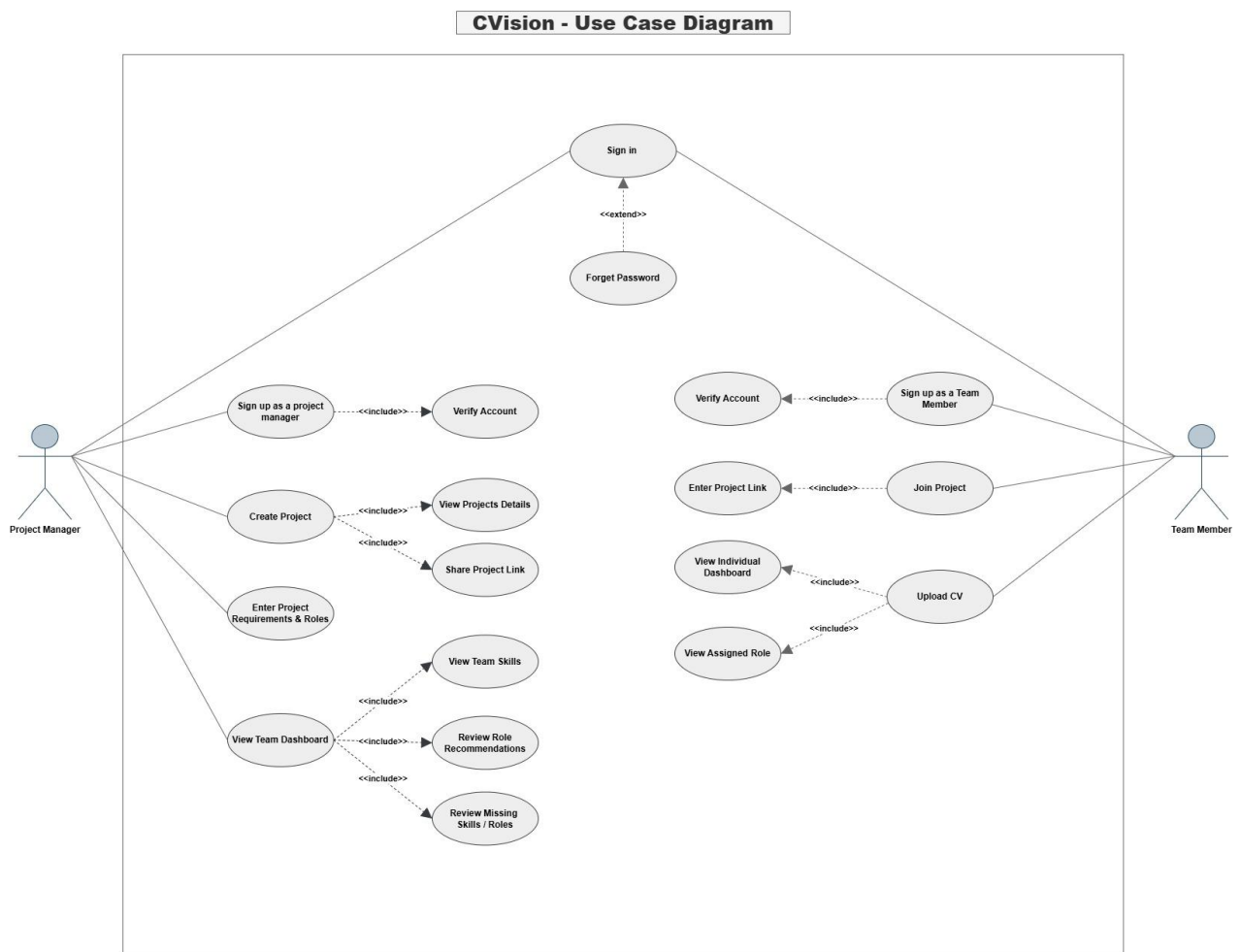


Figure 1: Use Case Diagram

UC-01 – Sign up as a Project Manager

Field	Description
Use Case ID	UC-01
Use Case Name	Sign up as a Project Manager
Primary Actor	Project Manager (new user)
Preconditions	1. User is not already registered in the system. 2. Registration service is available.
Postconditions	On Success: A new Project Manager account is created with role = “PM” and status = “Pending Verification”. On Failure: No account is created. An error message is displayed.
Main Flow	1. User opens the “Sign up as a Project Manager” page. 2. User enters required data (name, email, password, etc.). 3. User submits the registration form. 4. System validates the input. 5. System stores the new account with role “Project Manager”. 6. System triggers Verify Account (sends verification email / code).
Alternative Flows	A1 – Email already exists: At step 4, if the email is already in use, the system displays “ <i>This email is already registered.</i> ” and stops the use case. A2 – Weak or invalid password: At step 4, if the password does not meet security rules, the system requests the user to change it. A3 – System error while saving: At step 5, if the database is unavailable, the system shows a generic error message and no account is created.

Table 4: Sign up as a Project Manager

UC-02 – Sign up as a Team Member

Field	Description
Use Case ID	UC-02
Use Case Name	Sign up as a Team Member
Primary Actor	Team Member (new user)
Preconditions	1. User is not already registered. 2. Registration service is available.
Postconditions	Success: A new Team Member account is created with role = “Member” and status = “Pending Verification”. Failure: No account is created.
Main Flow	1. User opens the “Sign up as a Team Member” page. 2. User enters required data (name, email, password). 3. User submits the registration form. 4. System validates the input. 5. System stores the new account with role “Team Member”. 6. System triggers Verify Account (sends verification email / code).
Alternative Flows	A1 – Email already exists: At step 4, if the email is already in use, the system displays “<i>This email is already registered.</i>” and stops the use case. A2 – Weak or invalid password: At step 4, if the password does not meet security rules, the system requests the user to change it. A3 – System error while saving: At step 5, if the database is unavailable, the system shows a generic error message and no account is created.

Table 5: Sign up as a Team Member

UC-03 – Verify Account

Field	Description
Use Case ID	UC-03
Use Case Name	Verify Account
Primary Actor	Registered User (PM or Team Member)
Preconditions	1. User has a pending account (not verified). 2. User has received a verification link/code.
Postconditions	Success: Account status is updated to “Active/Verified”. Failure: Account remains “Pending Verification”.
Main Flow	1. User opens verification page or link. 2. User enters the verification code and submits. 3. System checks the code and account. 4. System confirms verification and activates the account.
Alternative Flows	A1 – Invalid code: System displays “ <i>Invalid verification code</i> ” and asks user to retry. A2 – Expired code: System shows “ <i>Verification code has expired, Request a new one.</i> ” A3 – Missing code: User submits empty field → system prompts user to enter a code.

Table 6: Verify Account

UC-04 – Sign in

Field	Description
Use Case ID	UC-04
Use Case Name	Sign in
Primary Actor	Project Manager, Team Member
Preconditions	1. User already has a registered account. 2. Authentication service is available.
Postconditions	Success: User is authenticated and redirected to the appropriate dashboard. Failure: User remains unauthenticated.
Main Flow	1. User opens the Sign in page. 2. User enters email and password. 3. System validates credentials. 4. System identifies user role. 5. If role = PM → redirect to Team Dashboard. 6. If role = Member → redirect to Individual Dashboard.
Alternative Flows	A1 – Wrong email/password: At step 3, if invalid, system shows <i>“Incorrect email or password.”</i> and allows retry. A2 – Account not verified: System shows <i>“Please verify your account first.”</i> and may redirect to Verify Account. A3 – Password forgotten: User chooses <i>“Forget Password”</i> → triggers UC-05.

Table 7: Sign in

UC-05 – Forget Password

Field	Description
Use Case ID	UC-05
Use Case Name	Forget Password
Primary Actor	User (PM or Team Member)
Preconditions	1. User has a registered email address.
Postconditions	Success: Reset password link/code is sent to the user's email. Failure: No reset link is sent.
Main Flow	1. From Sign in page, user selects "Forget Password". 2. User enters registered email. 3. System checks if email exists. 4. System generates password reset token and sends email.
Alternative Flows	A1 – Email not found: At step 3, system shows <i>"This email is not registered."</i> A2 – Email service error: At step 4, if email cannot be sent, system displays an error and suggests trying later.

Table 8: Forget Password

UC-06 – Create Project

Field	Description
Use Case ID	UC-06
Use Case Name	Create Project
Primary Actor	Project Manager
Preconditions	1. PM is signed in and verified.
Postconditions	Success: A new project record is stored. A Project link is generated. Failure: No project is created.
Main Flow	1. PM navigates to “Create Project”. 2. PM enters project name, description, and optional dates. 3. System validates the data. 4. System saves the project. 5. System executes Project link and links the link to this project. 6. System displays confirmation and the generated project link.
Alternative Flows	A1 – Missing required fields: At step 3, if project name or mandatory fields are empty, system prompts PM to complete them. A2 – Duplicate project name: System warns if another project with same name exists; PM may choose a different name. A3 – Database error: If saving fails, system shows an error message and cancels creation.

Table 9: Create Project

UC-07 – Enter Project Requirements & Roles

Field	Description
Use Case ID	UC-07
Use Case Name	Enter Project Requirements & Roles
Primary Actor	Project Manager
Preconditions	1. Target project exists. 2. PM is signed in and owns/has access to the project.
Postconditions	Success: Project is updated with roles and required skills (with weights). Failure: No changes are saved.
Main Flow	1. PM opens the project's "Requirements & Roles" page. 2. PM adds one or more roles (e.g.: Programmer, Designer, Tester). 3. For each role, PM specifies required skills and their importance (weights). 4. PM saves the configuration. 5. System stores all roles and skills for this project.
Alternative Flows	A1 – No roles specified: System prompts <i>"At least one role is required."</i> and does not save. A2 – Invalid or empty skill fields: System marks invalid entries and asks PM to fix them.

Table 10: Enter Project Requirements & Roles

UC-08 – View Team Dashboard

Field	Description
Use Case ID	UC-08
Use Case Name	View Team Dashboard
Primary Actor	Project Manager
Preconditions	1. PM is signed in. 2. At least one project exists.
Postconditions	Dashboard view is displayed; no data is modified.
Main Flow	1. PM selects a project and opens the Team Dashboard. 2. System retrieves team members, their skills, and role information. 3. System also triggers internal includes: View Team Skills (UC-10), Generate Role Recommendations (UC-11), and Review Missing Skills / Roles (UC-13) to enrich the view. 4. System displays charts, tables, and alerts summarizing team status.
Alternative Flows	A1 – No team members joined yet: System shows message <i>“No team members have joined this project yet.”</i> A2 – CVs still being processed: For some members, skills may be marked as <i>“Processing”</i> .

Table 11: View Team Dashboard

UC-9 – View Team Skills

Field	Description
Use Case ID	UC-9
Use Case Name	View Team Skills
Primary Actor	Project Manager
Preconditions	1. PM has at least one team member joined to the project. 2. Members have uploaded CVs.
Postconditions	A list/summary of each member's skills and scores is shown.
Main Flow	1. System (or PM) requests skills overview for a project. 2. System retrieves all extracted skills for members linked to the project. 3. System displays skills per member with corresponding Skill Scores.
Alternative Flows	A1 – No CVs uploaded: System shows <i>"No CV data available. Ask team members to upload their CVs."</i>

Table 12: View Team Skills

UC-10 – Review Role Recommendations

Field	Description
Use Case ID	UC-10
Use Case Name	Review Role Recommendations
Primary Actor	Project Manager
Preconditions	1. Project has defined roles & required skills (UC-07). 2. Team members with extracted skills exist for the project.
Postconditions	Success: Role–member recommendations and matching scores are produced. Failure: No recommendations generated.
Main Flow	1. PM (or system via dashboard) triggers generation of recommendations. 2. System compares each member’s skills and scores with each role’s requirements. 3. System computes matching scores and ranks members for each role. 4. System displays top candidates for each role.
Alternative Flows	A1 – Requirements missing: If no roles are defined, system shows <i>“Define project roles before generating recommendations.”</i> A2 – No suitable candidates: For some roles, system shows <i>“No candidate meets minimum criteria.”</i>

Table 13: Project Manager

UC-11 – Review Missing Skills / Roles

Field	Description
Use Case ID	UC-11
Use Case Name	Review Missing Skills / Roles
Primary Actor	Project Manager
Preconditions	1. Project requirements (roles + skills) exist. 2. Some member skills have been analyzed.
Postconditions	A list of missing/weak skills and roles is displayed; no data is changed.
Main Flow	1. System compares project requirements with the current team's skills. 2. System identifies skills that are missing or below threshold, and roles with no suitable candidates. 3. System displays a summary to the PM (e.g. : No Designer with "UI/UX", Weak skill: Database Design").
Alternative Flows	A1 – No missing skills: System shows <i>"All required skills are currently covered."</i>

Table 14: Review Missing Skills / Roles

UC-12 – Join Project

Field	Description
Use Case ID	UC-12
Use Case Name	Join Project
Primary Actor	Team Member
Preconditions	1. Team Member is signed in and verified. 2. A valid Project link exists for the target project.
Postconditions	Success: Team Member becomes associated with the selected project. Failure: No association is created.
Main Flow	1. Team Member selects “Join Project”. 2. System prompts for a Project link. 3. TM enters the link (includes Enter Project link-UC-13). 4. System validates the code and links the member to the project. 5. System confirms successful joining.
Alternative Flows	A1 – Invalid code: System shows “Invalid Project link.” A2 – Expired/disabled code: System shows “This Project link is no longer active.” A3 – Member already part of project: System shows “You are already a member of this project.”

Table 15: Join Project

UC-13 – Enter Project link

Field	Description
Use Case ID	UC-13
Use Case Name	Enter Project link
Primary Actor	Team Member
Preconditions	1. Team Member is attempting to join a project (UC-12).
Postconditions	Success: A Project link value is provided for validation. Failure: Project link cannot proceed.
Main Flow	1. System displays an input field for the Project link. 2. TM types the received Project link and submits.
Alternative Flows	A1 – Empty code: System requests the user to enter a project link.

Table 16: Enter Project link

UC-14 – Upload CV

Field	Description
Use Case ID	UC-14
Use Case Name	Upload CV
Primary Actor	Team Member
Preconditions	1. TM is signed in. 2. TM belongs to at least one project (joined via Project link).
Postconditions	Success: CV is stored, processed via gemini ai, and skills & experience are extracted and stored in the member's profile. Failure: No skills are extracted; previous data (if any) remain unchanged.
Main Flow	1. TM opens the "Upload CV" page. 2. TM selects a CV file (PDF/Word). 3. TM confirms upload. 4. System uploads and reads the file. 5. System applies gemini ai to extract skills, tools, and years of experience. 6. System calculates a Skill Score for each skill and saves results.
Alternative Flows	A1 – Unsupported file type: System rejects the file and displays <i>"Only PDF and Word files are allowed."</i> A2 – File too large: System shows <i>"File size exceeds the allowed limit."</i> A3 – Processing error: Gemini AI extraction fails → system shows <i>"Could not process the CV. Please try again."</i>

Table 17: Upload CV

UC-15 – View Individual Dashboard

Field	Description
Use Case ID	UC-15
Use Case Name	View Individual Dashboard
Primary Actor	Team Member
Preconditions	1. TM is signed in. 2. At least one CV has been successfully processed, or the system must handle “no data” case.
Postconditions	Dashboard is shown; no stored data are changed.
Main Flow	1. TM opens “My Dashboard / Individual Dashboard”. 2. System retrieves the member’s extracted skills, scores, and experience. 3. System displays charts and lists for: strengths, low skills, and improvement suggestions. 4. System also shows the Assigned Role via UC-16 (include).
Alternative Flows	A1 – No CV uploaded/processed: System shows message “ <i>No analyzed CV available. Please upload your CV.</i> ” and may hide skill charts.

Table 18: View Individual Dashboard

UC-16 – View Assigned Role

Field	Description
Use Case ID	UC-16
Use Case Name	View Assigned Role
Primary Actor	Team Member
Preconditions	1. TM belongs to at least one project. 2. Roles have been assigned (automatically or manually) by the Project Manager.
Postconditions	The assigned role(s) are displayed to the member.
Main Flow	1. As part of the Individual Dashboard, the system retrieves the member's current role assignment for the selected project. 2. System displays the role (e.g. : "Programmer", "Designer") and possibly the project name.
Alternative Flows	A1 – No role assigned yet: System shows <i>"Role assignment is not completed yet."</i> or similar notice.

Table 19: View Assigned Role

3.3 Non-Functional User Requirements

Non-functional requirements define the quality attributes of CVision, ensuring performance, reliability, maintainability, and scalability. They are categorized into execution qualities (observable at runtime) and evolution qualities (affect system structure and future adaptability).

3.3.1 Execution Qualities

Usability

- The system must provide a simple and user-friendly interface for uploading CVs and navigating system features.
- Dashboards, charts, and visual elements should be clear and easy to understand.

Performance

- The system should process uploaded CVs within a reasonable time under normal usage conditions.
- Dashboards and project pages should load quickly and provide a smooth user experience.

Security

- CV files and user data must be stored securely and accessed only by authorized users.
- User authentication must follow secure practices, including password hashing and protected login mechanisms.
- Project management actions must only be accessible to authorized Project Managers.

Reliability

- The system should operate reliably during normal usage and handle unexpected errors without losing user data.

- Error-handling mechanisms should provide meaningful feedback and support system recovery when possible.

Compatibility

- The system must function correctly on modern web browsers and different screen sizes.
- CVision must support PDF and DOCX file formats for CV uploads.

3.3.2 Evolution Qualities

Maintainability

- The system must follow a modular architecture that separates the frontend, backend, AI integration, and database components.
- The source code should follow clear naming conventions and proper documentation to simplify future maintenance.

Scalability

- The system should support an increasing number of users, projects, CV uploads, and skill records without significant performance degradation.
- The database structure should allow new roles, projects, and skills to be added easily.

Extensibility

- Future features and integrations should be added without requiring major changes to the core system architecture.
- The AI service can be updated or replaced with newer models when needed.

Testability

- The system should support testing of core functionalities, including CV uploads, AI analysis, role matching, and skill gap analysis.
- Test cases should cover different CV formats and user scenarios to ensure consistent system behavior.

3.3.3 Constraints

Technical Constraints

- The accuracy of AI-based CV analysis depends on the quality, completeness, and structure of the uploaded CV.
- The system currently supports English-language CVs in PDF and DOCX formats only.
- AI analysis depends on the availability and usage limits of external AI services.

Operational Constraints

- Users must have a stable internet connection to upload CVs and access system features.
- The hosting environment must support the technologies used in the project, including FastAPI, React, PostgreSQL, and AI service integration.

Regulatory Constraints

- The system must comply with academic and institutional requirements regarding the storage, processing, and protection of personal data contained in uploaded CVs.

CHAPTER 4: ARCHITECTURE AND DESIGN

4.1 Overview

This chapter presents the architectural and design aspects of the CVision system. It describes how the system components are structured, how they interact with each other, and how the overall design supports the functional and non-functional requirements defined in Chapter 3. The chapter covers the system architecture, detailed software design using UML diagrams, and the user interface design through prototypes.

4.2 Software Architecture

4.2.1 Logical view

In this section, we will present the logical view of software architecture using a component diagram or class diagram (UML). This representation illustrates the main components of the system and their relationships, making it easier to understand how they interact to achieve the system's functionalities. **we use Draw.io**

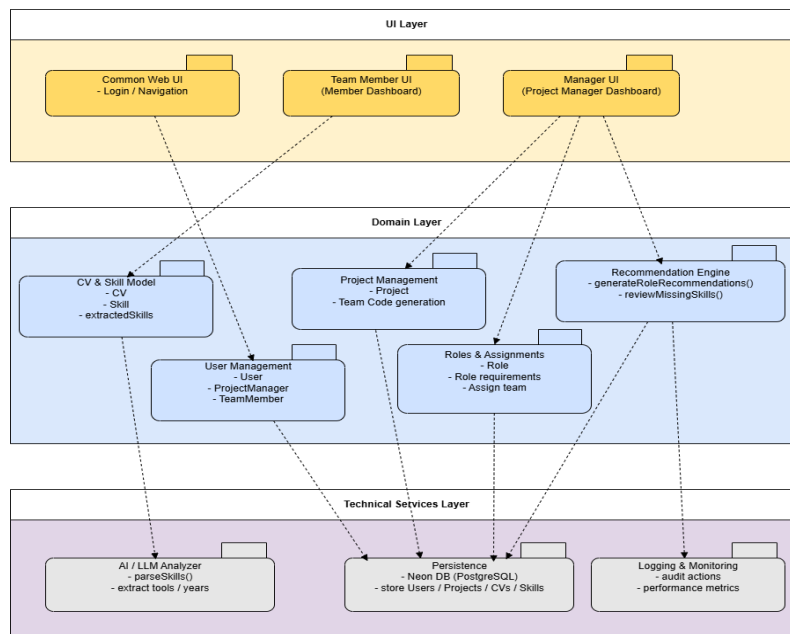


Figure 2: Logical View Diagram

4.2.2 Process view

In this section, we will present the process view of software architecture using a process diagram, illustrating how processes are executed and the interactions between components to achieve the desired objectives.

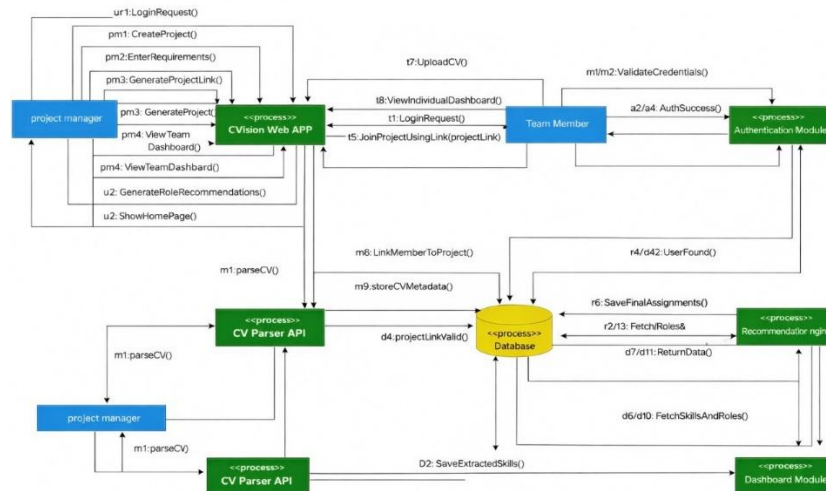


Figure 3: Process Diagram

4.2.3 Physical view

In this section, we will present the physical view of software architecture using a deployment diagram, illustrating the distribution of physical components and the infrastructure of the system.

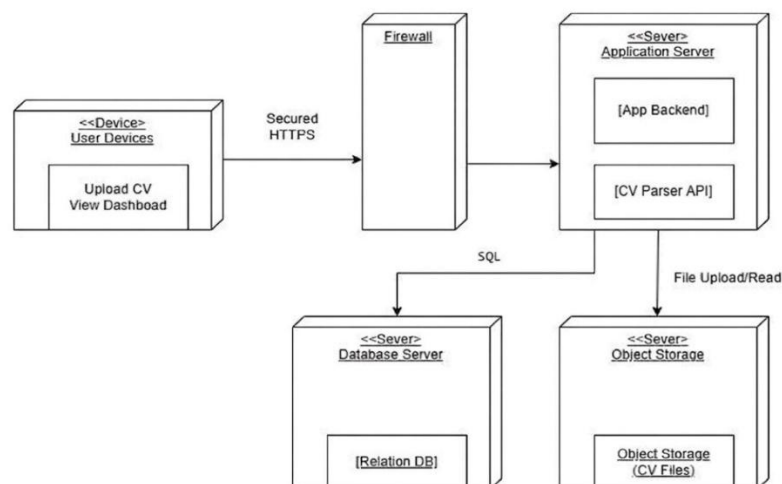


Figure 4: Physical Diagram

4.2.4 Details of each component in a separate section.

This section describes each main component of the CVision software architecture. The description explains the responsibility of every component and its role within the overall system, as illustrated in the architecture diagrams.

User Devices

This component represents the devices used by end users, such as laptops or personal computers. Users access the CVision system through a web browser to upload CVs, view dashboards, and interact with system features.

Firewall

The firewall component provides a security layer that protects the system from unauthorized access. It ensures that only secure and permitted requests are forwarded to the application server.

Application Server

The application server hosts the main system services. It receives requests from user devices, processes business logic, and coordinates communication between backend modules, the database, and storage services.

App Backend

The App Backend component contains the core business logic of CVision. It handles user authentication, project management, CV upload requests, role definitions, and access control. This component acts as the central controller of the system.

CV Parser API

A specialized service that extracts and structures information from uploaded CV files (e.g., parsing names, skills, experience), then routes parsed data to the database and raw files to object storage.

Database Server (Relational Database)

The database server stores all structured system data. This includes user accounts, projects, team codes, extracted skills, proficiency scores, and final role assignments. It ensures data consistency and availability for all system components.

Object Storage (CV Files)

The object storage component stores uploaded CV files separately from the relational database. This separation improves performance and simplifies file management while ensuring secure access to CV documents.

Authentication Module

The authentication module manages user login, registration, and account verification. It ensures that only authorized users can access the system and that each user is directed at the appropriate interface based on their role.

Recommendation Engine

The recommendation engine analyzes extracted skill data and project requirements to generate role recommendations. It ranks candidates for each role, detects conflicts, and supports manual override by the project manager when needed.

Dashboard Module

The dashboard module generates visual summaries of individual and team data. It displays skill distributions, role assignments, overlaps, and missing skills using charts and tables to support informed decision-making.

Logging and Monitoring Module

This component records system events, user actions, and performance metrics. It supports auditing, debugging, and system monitoring to ensure reliability and maintainability.

4.3 Software design

4.3.1 UML sequence diagram

In this section, we will explain the sequence diagram, which illustrates the interactions between objects in the system and how they exchange messages. This diagram helps in understanding the flow of processes and the relationships between objects to achieve the desired functionalities. **We use Draw.io**

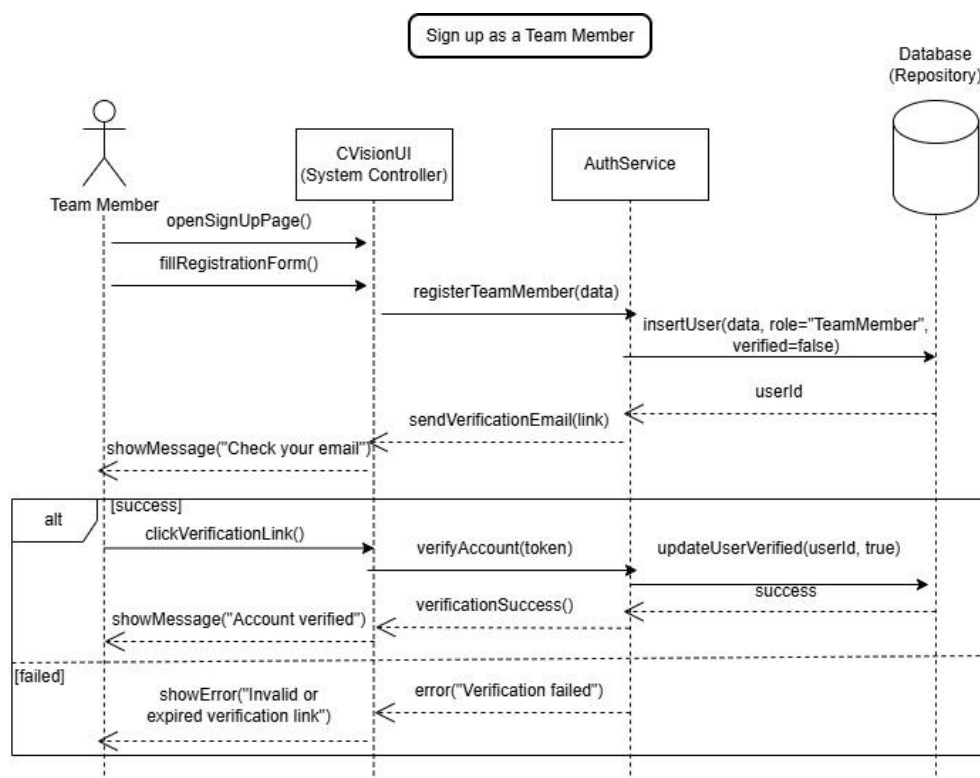


Figure 5: Sign up as a team member

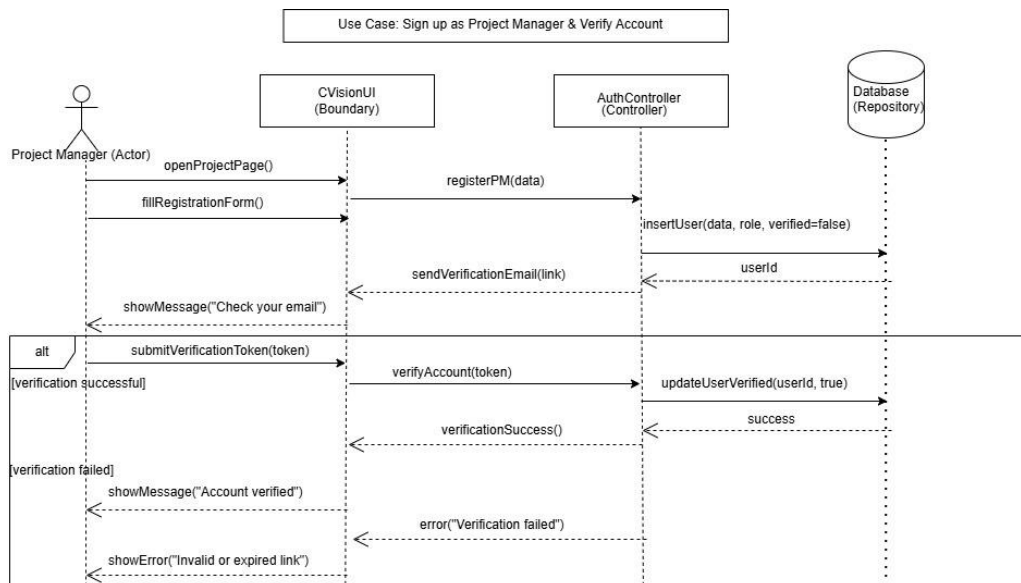


Figure 6: Sign up as a project manager & verify account

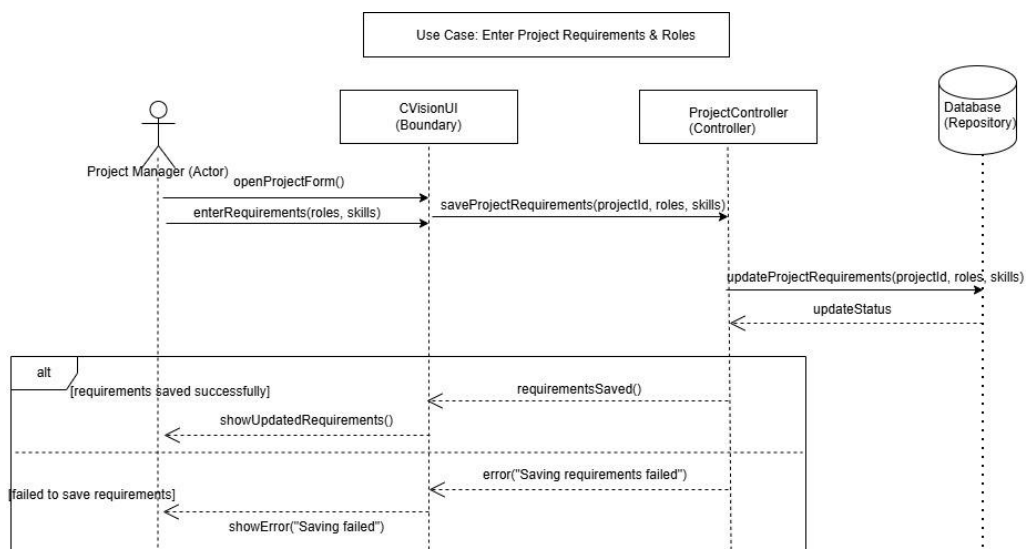


Figure 7: Enter project requirements & roles

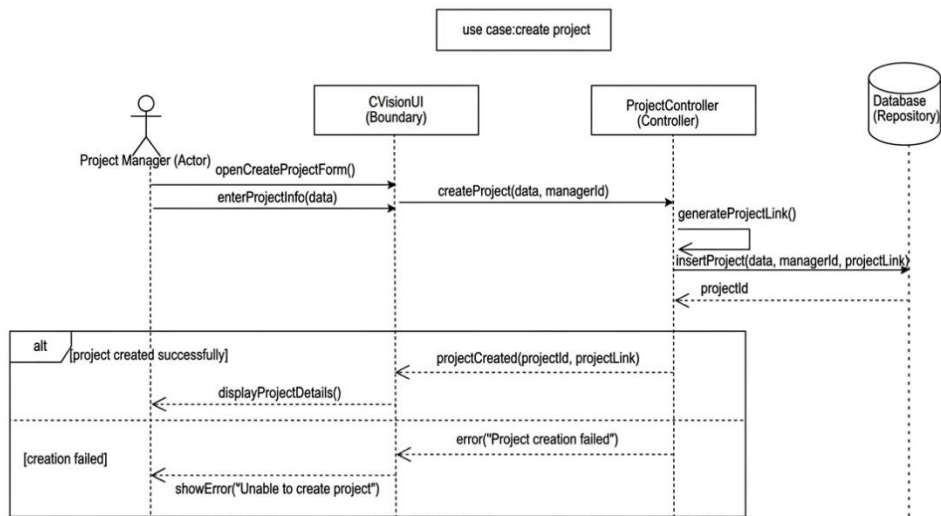


Figure 8: Create project

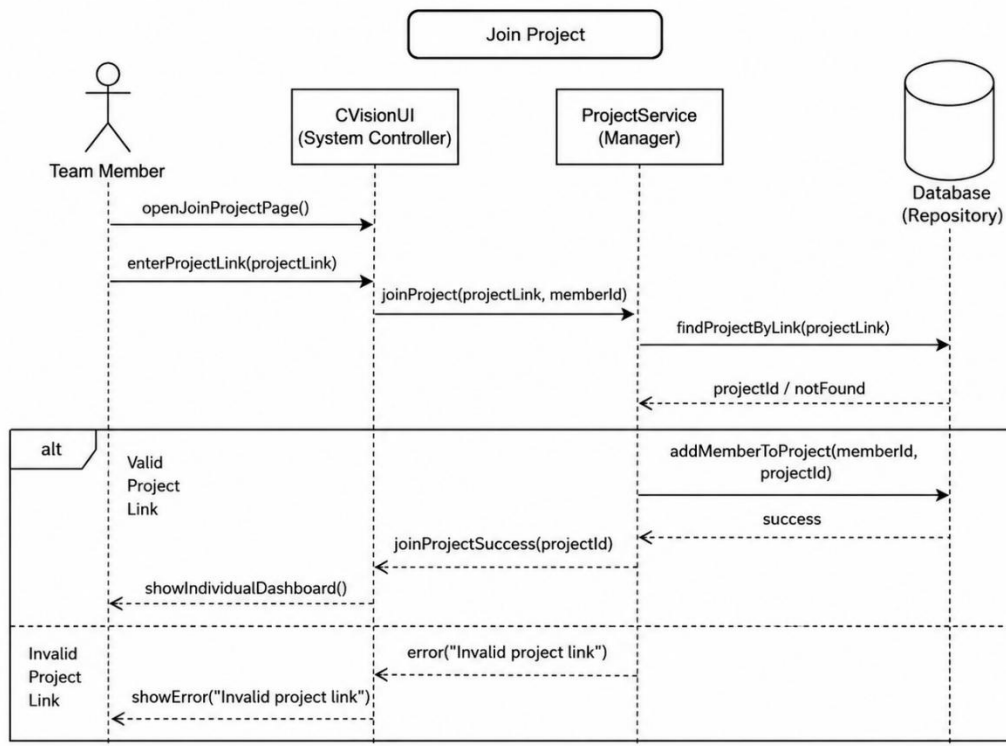


Figure 9: Join Project

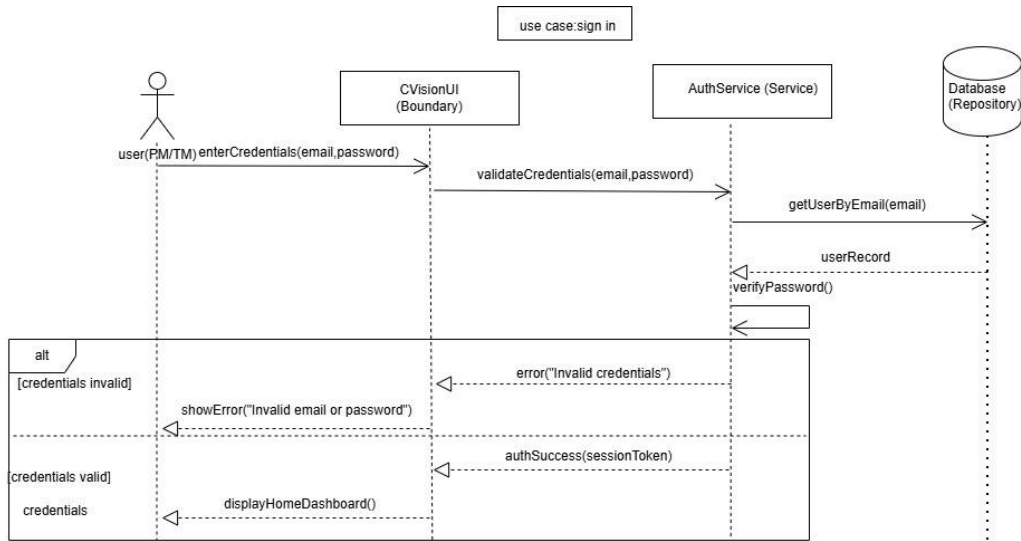


Figure 10: Sign in

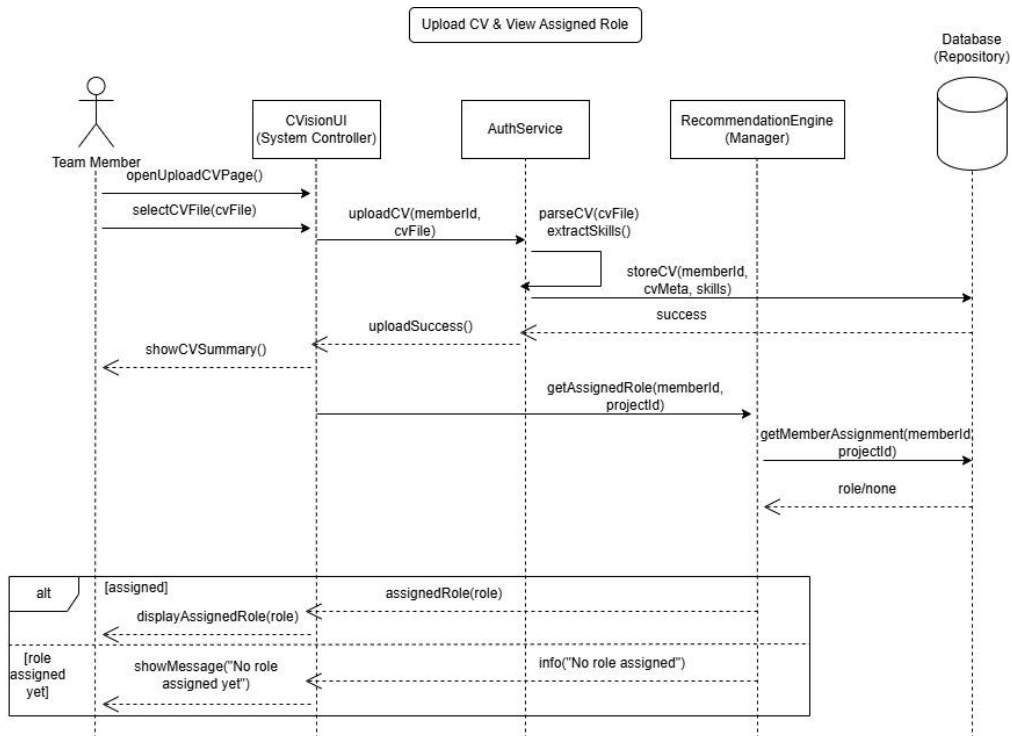


Figure 11: Upload CV & View assigned role

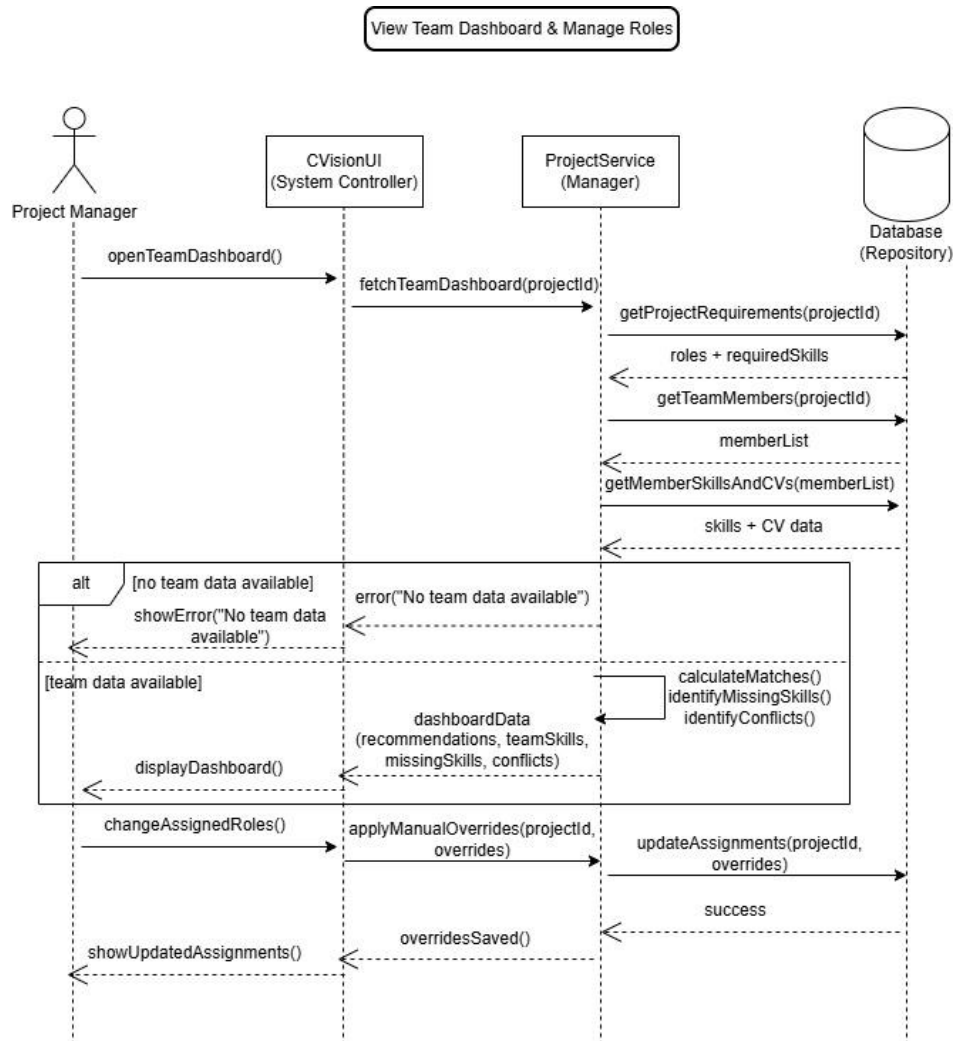


Figure 12: Enter project requirements & roles

4.3.2 Class diagram

In this section, we will review the class diagram, which is used to illustrate the structure of the system by defining the classes, attributes, methods, and the relationships between them. The class diagram helps organize the code and clarifies how objects interact within the system. This diagram presents details such as inheritance and composition, making it easier to understand object-oriented design and assisting developers in building the system more effectively. **we use Draw.io**

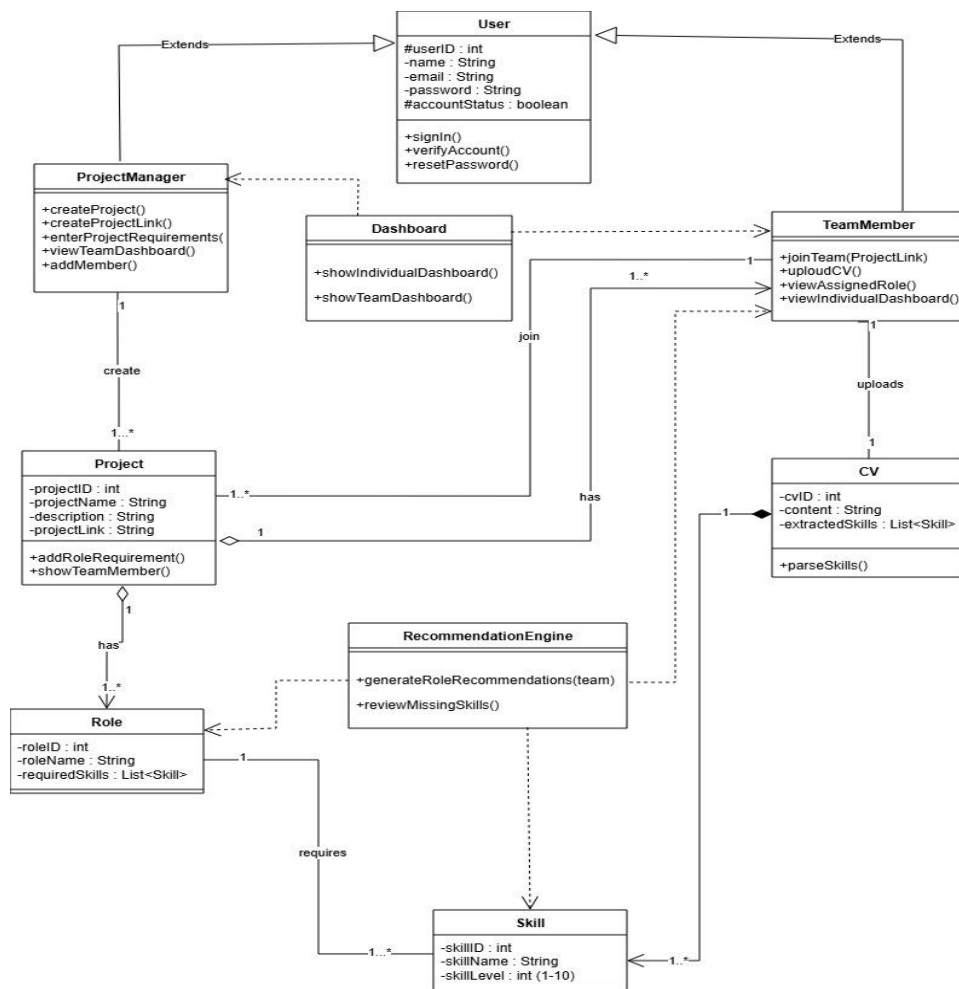


Figure 13: Class Diagram

4.3.3 ER diagram

In this section, we will review the Entity-Relationship diagram (ER Diagram), which is used to illustrate the database structure. The diagram shows the different entities in the system, such as tables or elements, and the relationships between them, making it easier to understand how data is organized and interacts. The ER diagram helps in effectively designing the database and ensuring it meets the system's needs. **we use Draw.io**

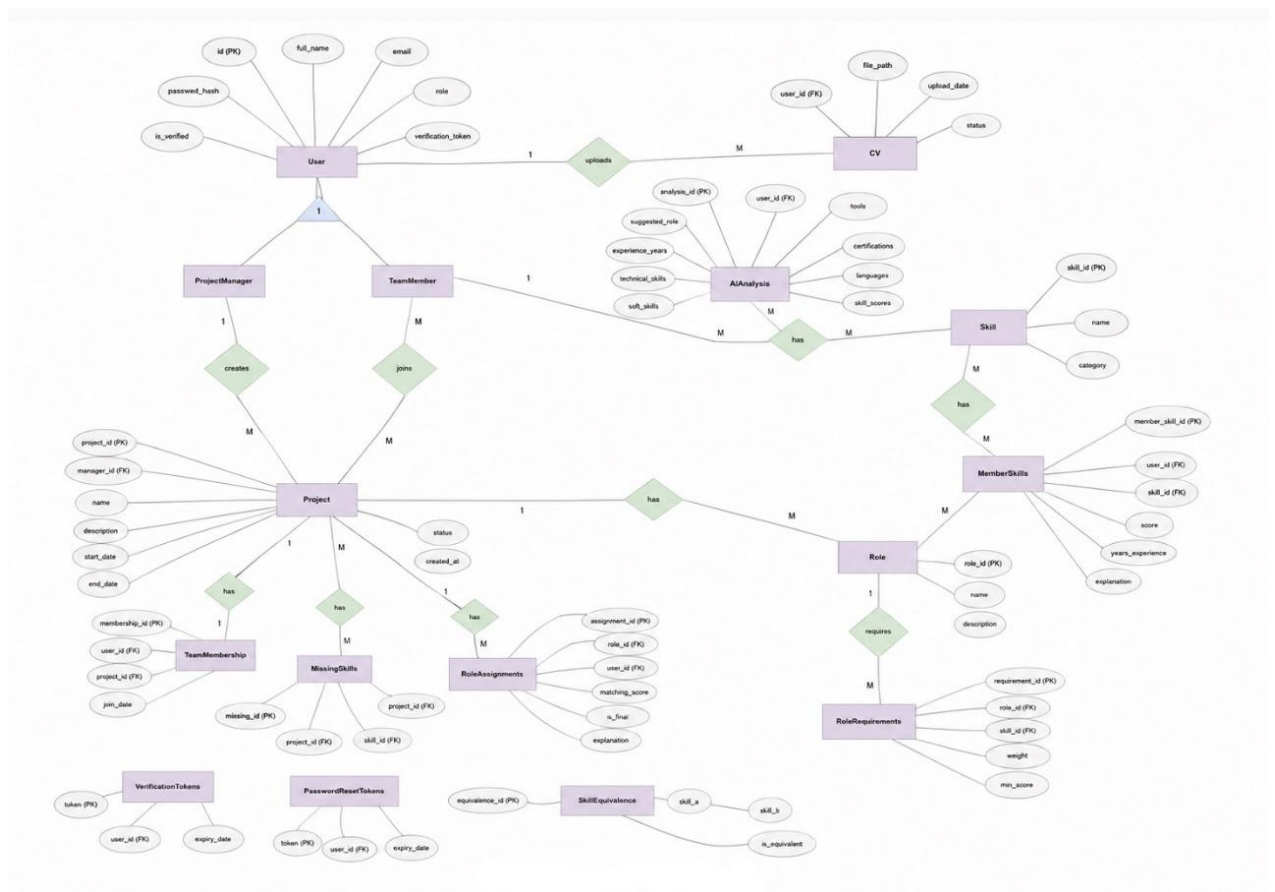


Figure 14: ER Diagram

4.3.4 State transition diagram

In this section, we will review the state transition diagram, which illustrates how the state of the system changes in response to events. The diagram shows the different states and the transitions between them, helping us to understand the dynamics of the system and how components interact with events. **we use Draw.io**

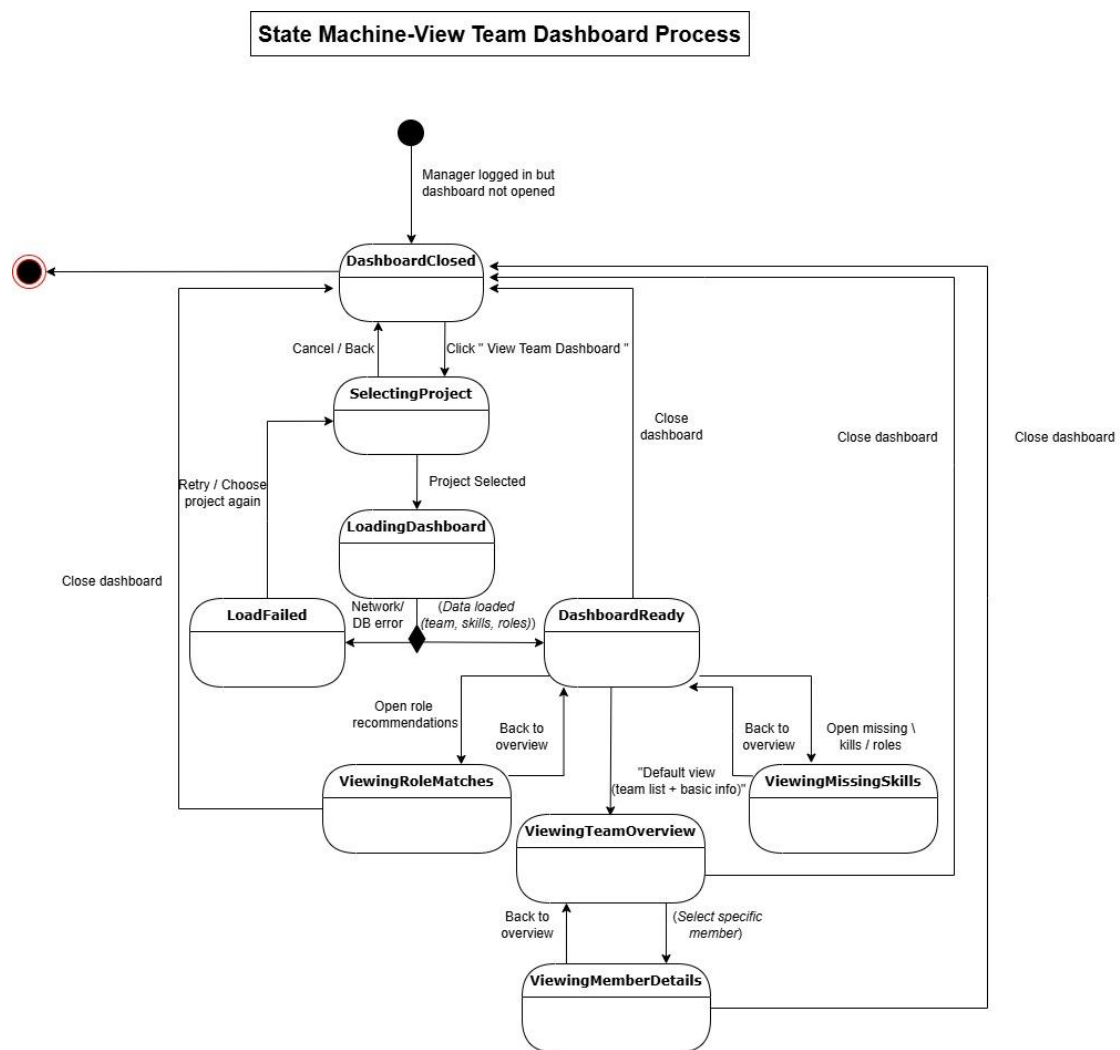


Figure 15: View team dashboard process

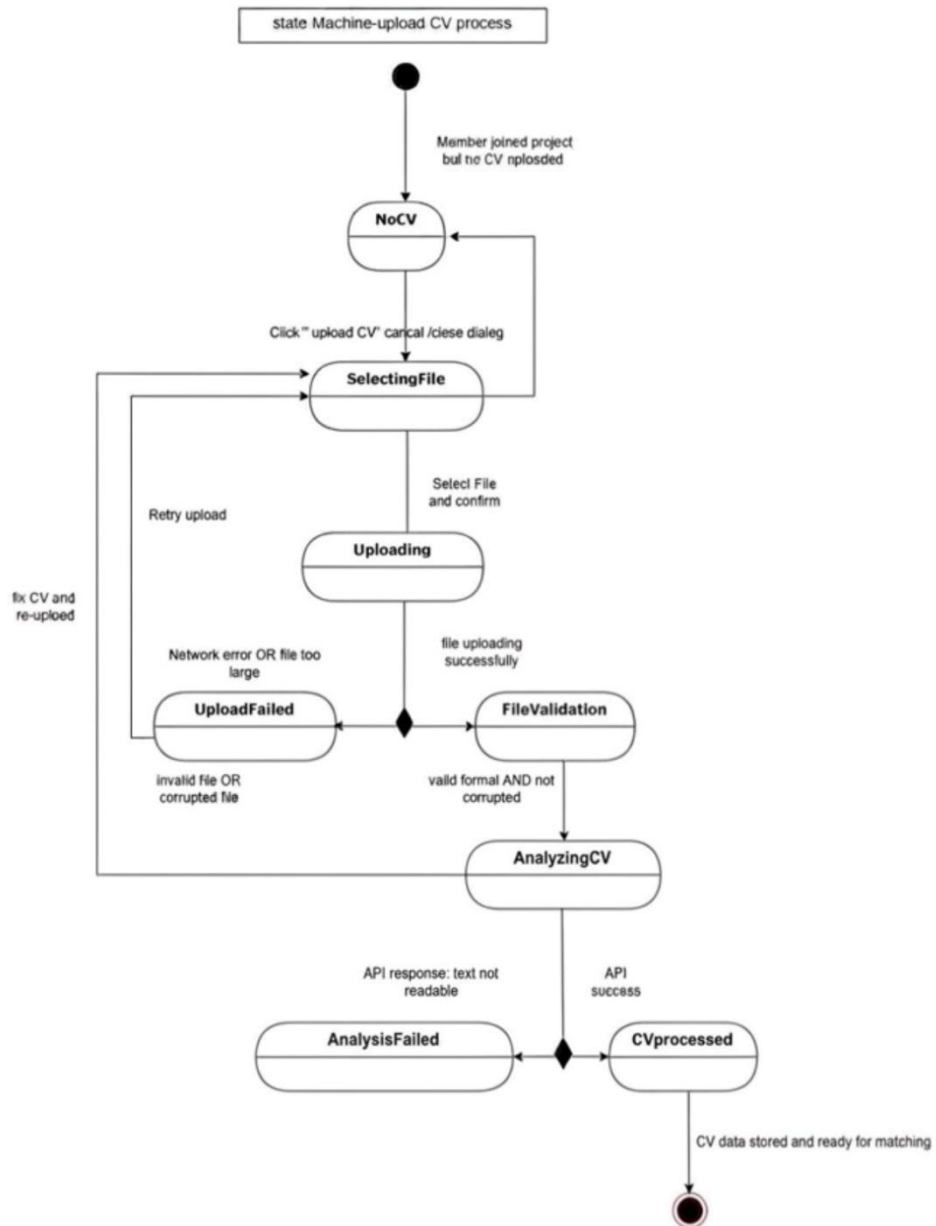


Figure 16: Upload CV process

4.4 User interface design

This section presents the implemented user interface of the CVision system. The frontend was developed to provide an intuitive and efficient user experience, allowing users to interact with the system through well-structured forms, buttons, menus, and other interface components. The design emphasizes usability, accessibility, and responsiveness to ensure smooth interaction across different devices. The implemented interface reflects the system requirements and supports users in completing their tasks effectively and efficiently.

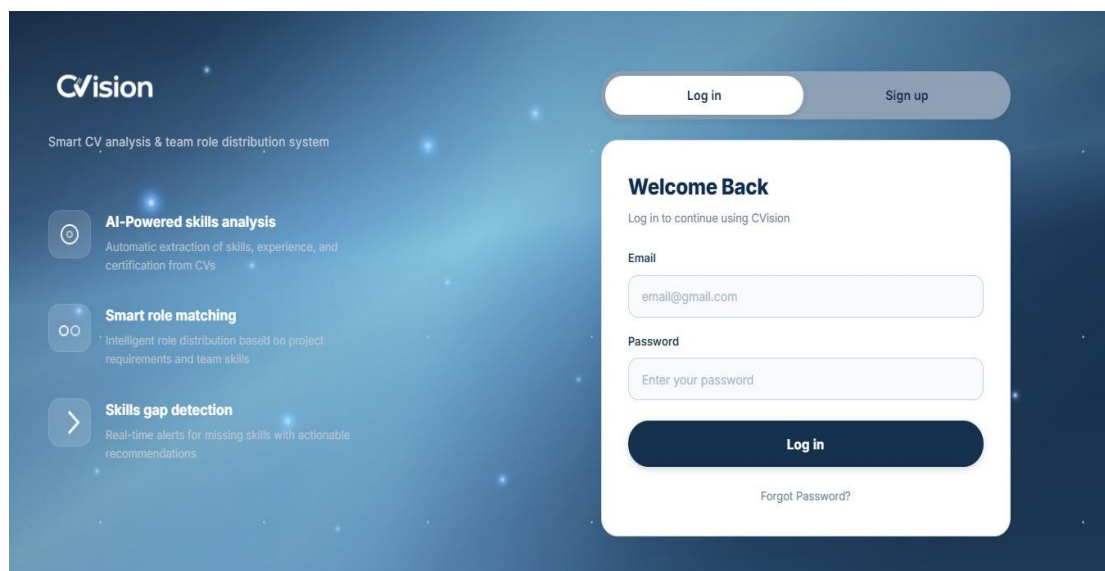


Figure 17: Log in

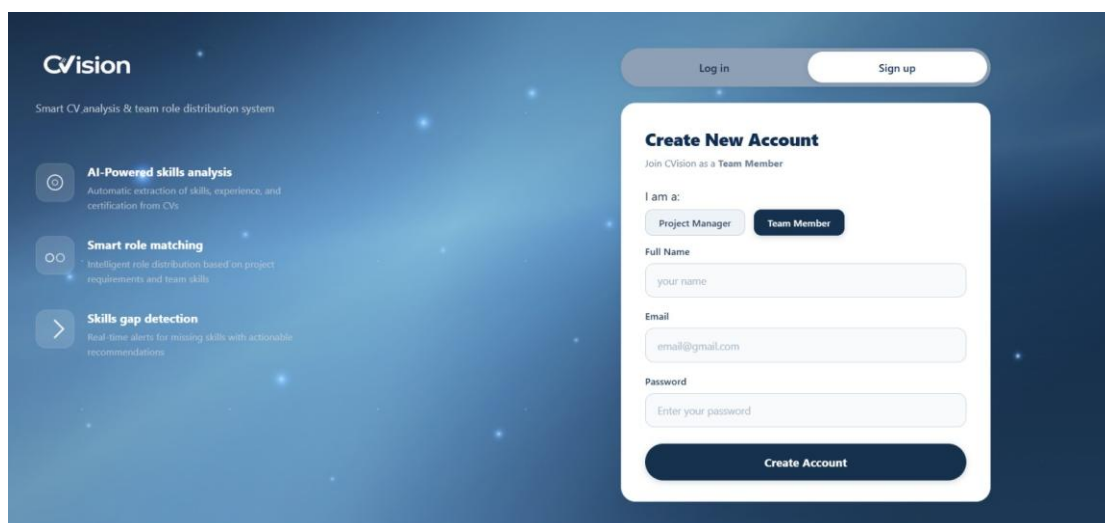


Figure 18: Sign up as a team member

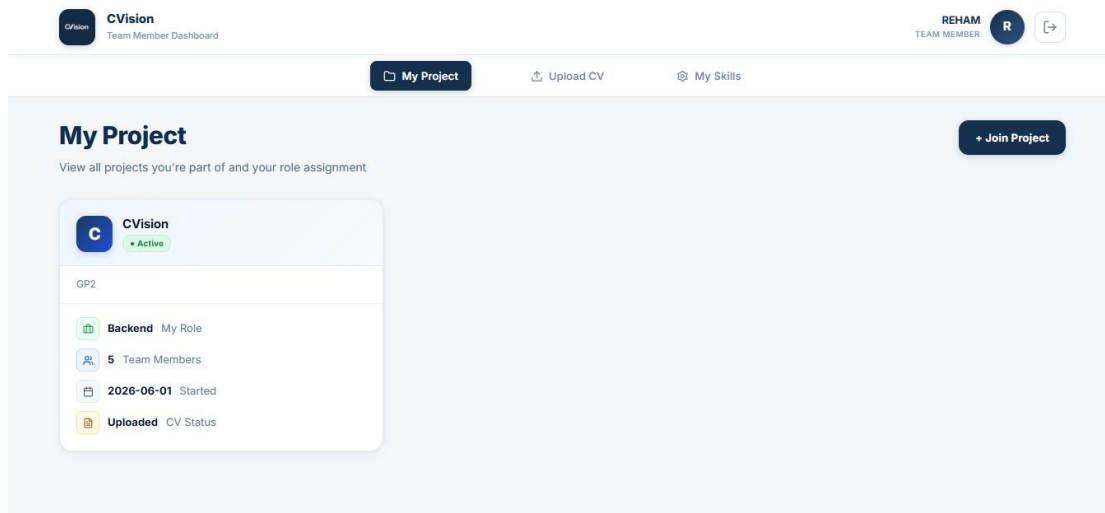


Figure 19: My project-TM

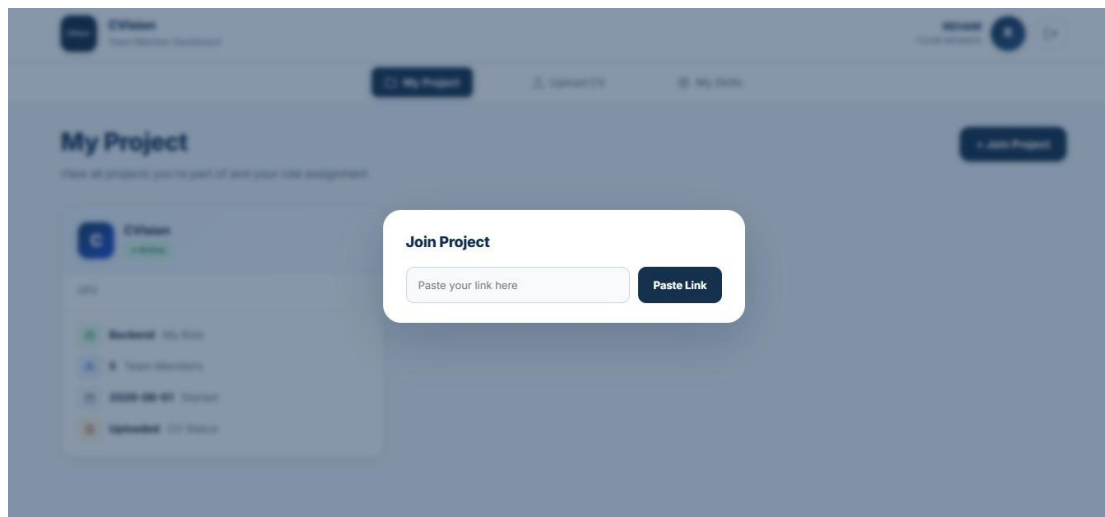


Figure 20: Join project

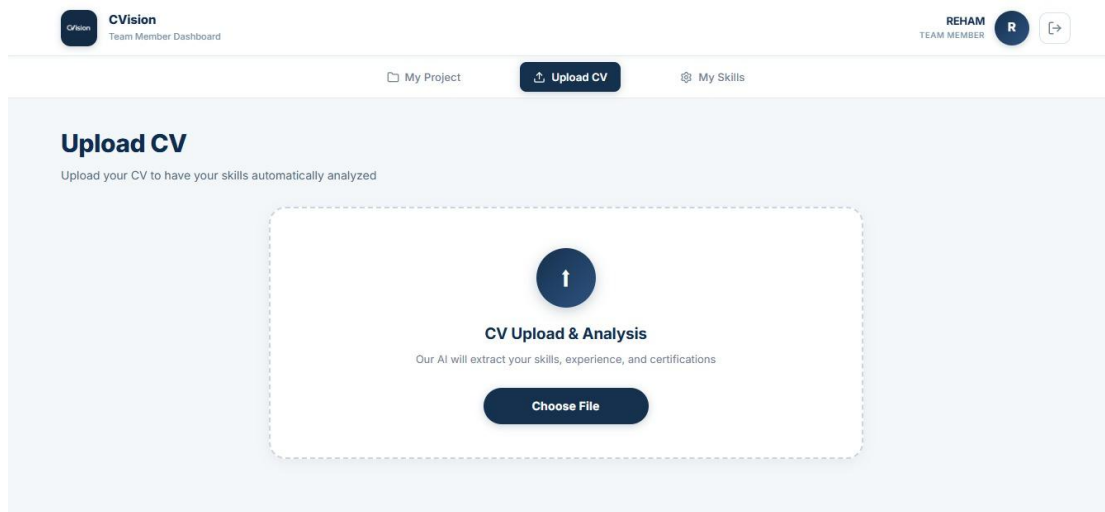


Figure 21: Upload CV

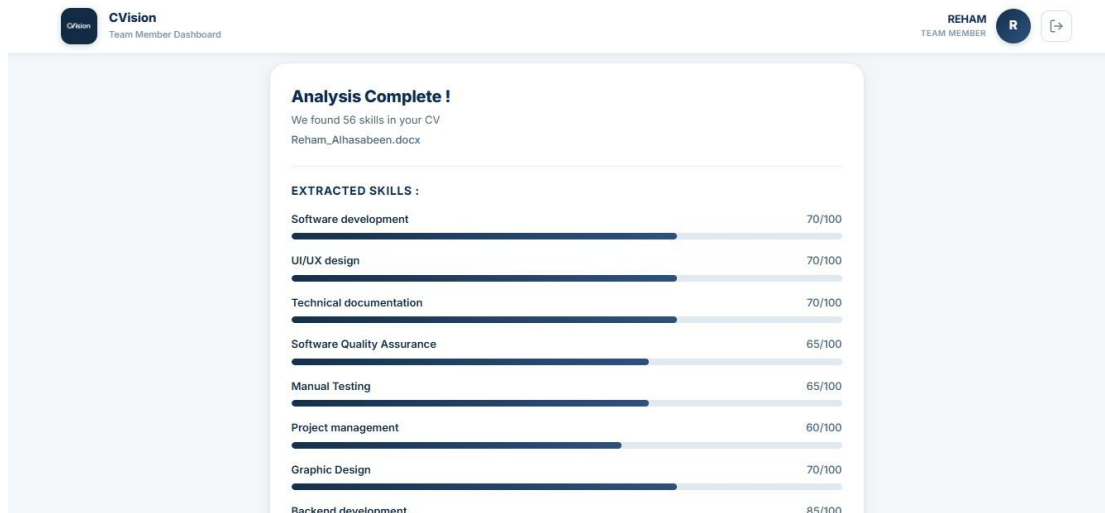


Figure 22: CV Analysis Complete

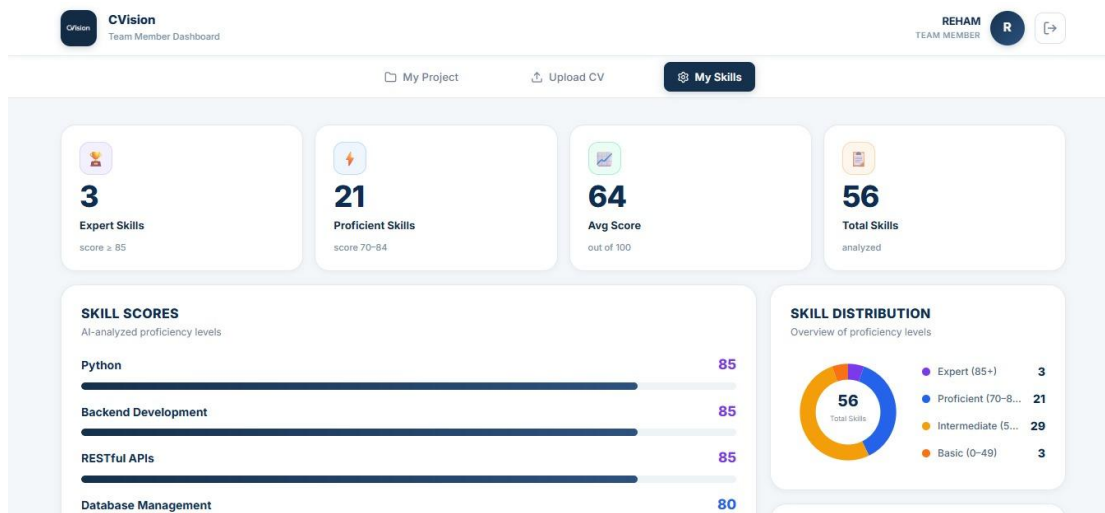


Figure 23: My Skill

CVision
Smart CV analysis & team role distribution system

Log in Sign up

Create New Account
Join CVision as a Project Manager

I am a:
☒ Project Manager
 ☐ Team Member

Full Name

Email

Password

Create Account

AI-Powered skills analysis
Automatic extraction of skills, experience, and certification from CVs

Smart role matching
Intelligent role distribution based on project requirements and team skills

Skills gap detection
Real-time alerts for missing skills with actionable recommendations

Figure 24: Sign up as a project manager

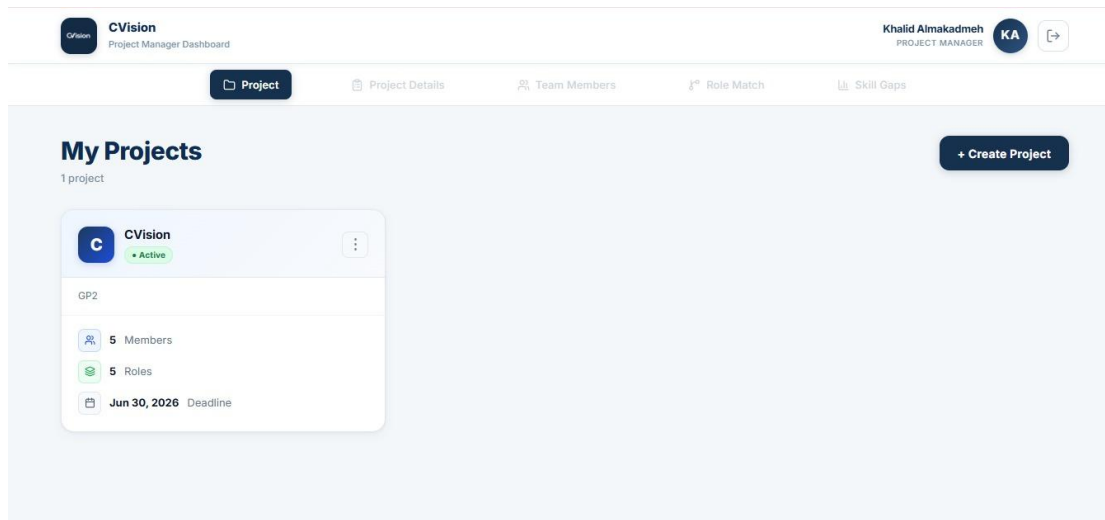


Figure 25: My Project-PM

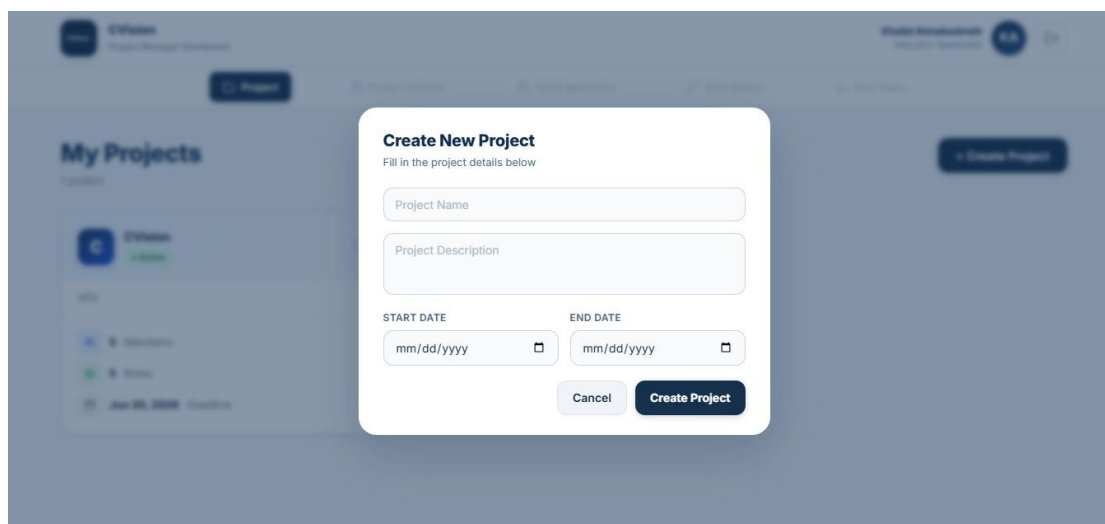


Figure 26: Create New Project

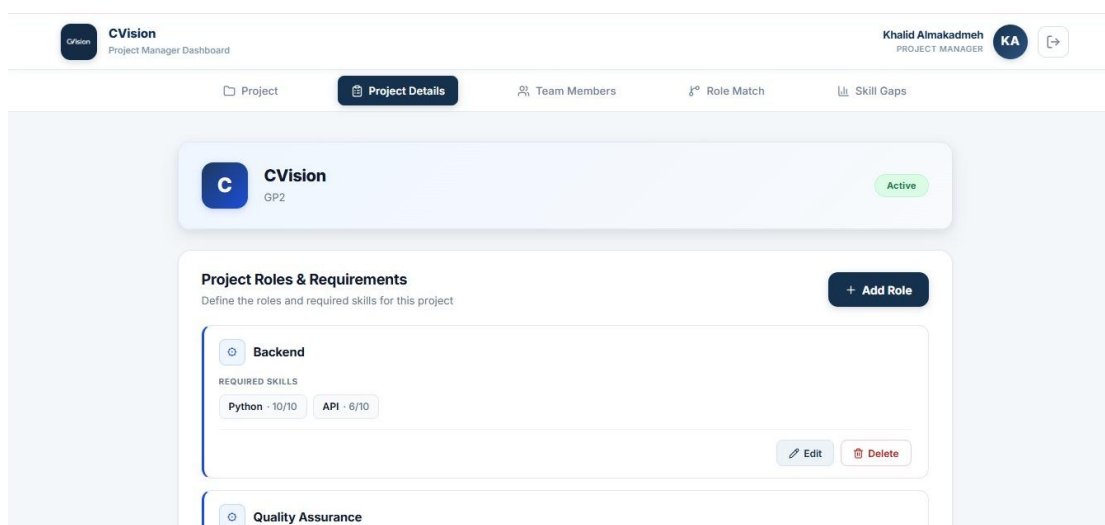
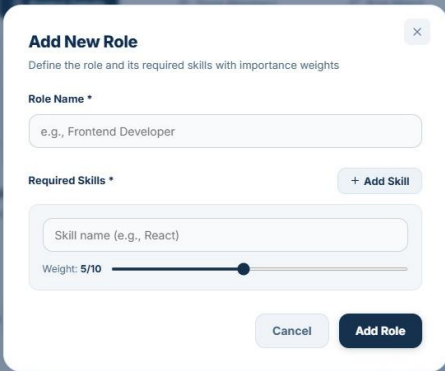


Figure 27: Project Details



Add New Role
Define the role and its required skills with importance weights

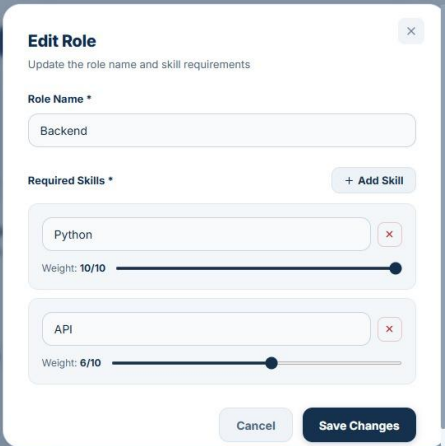
Role Name *
e.g., Frontend Developer

Required Skills * + Add Skill

Skill name (e.g., React)
Weight: 5/10

Cancel Add Role

Figure 28: Add New Role



Edit Role
Update the role name and skill requirements

Role Name *
Backend

Required Skills * + Add Skill

Python ×
Weight: 10/10

API ×
Weight: 6/10

Cancel Save Changes

Figure 29: Edit Role

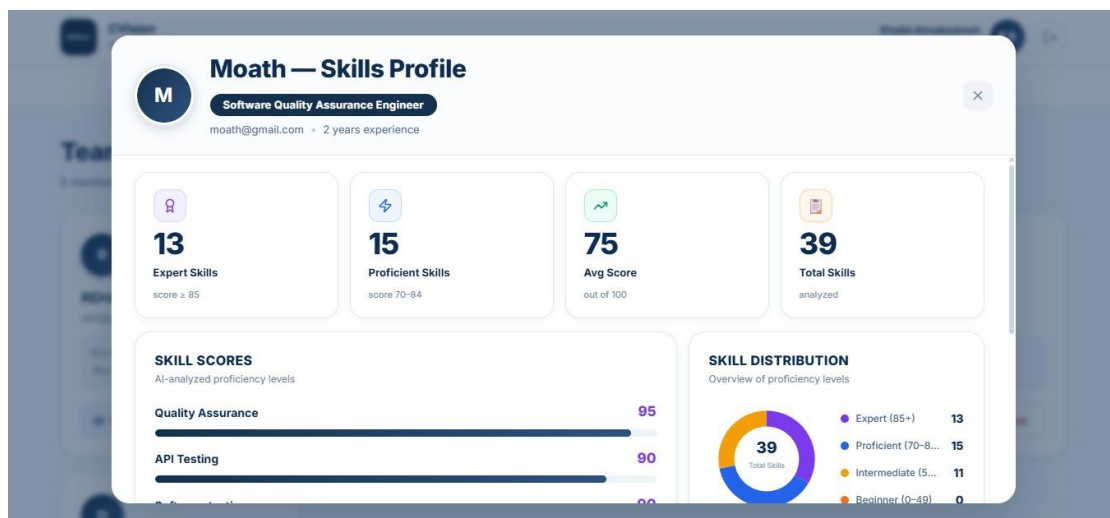


Figure 30: Skills Profile For Member

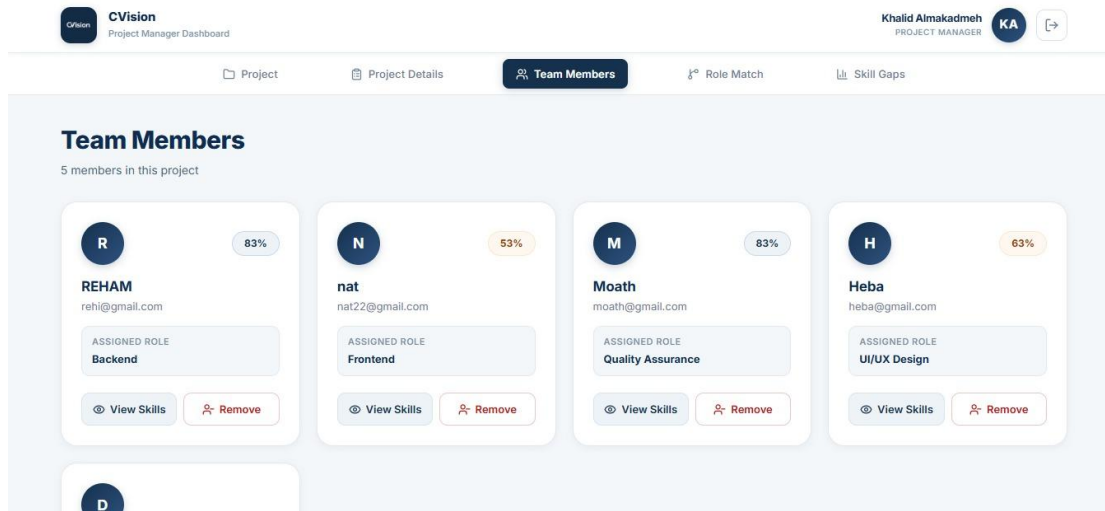


Figure 31: Team Members



Figure 32: Role Matching & Assignment

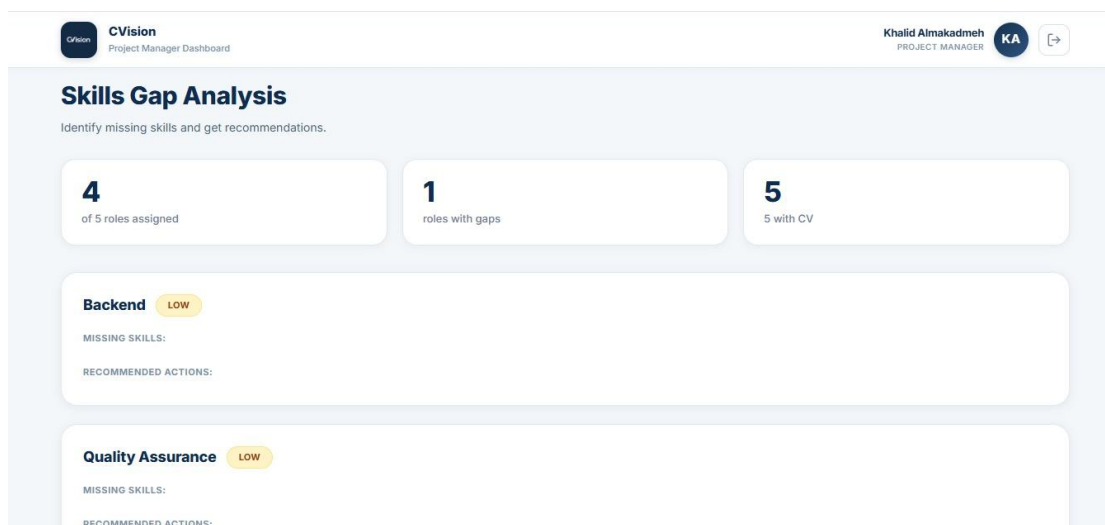


Figure 33: Skills Gap Analysis

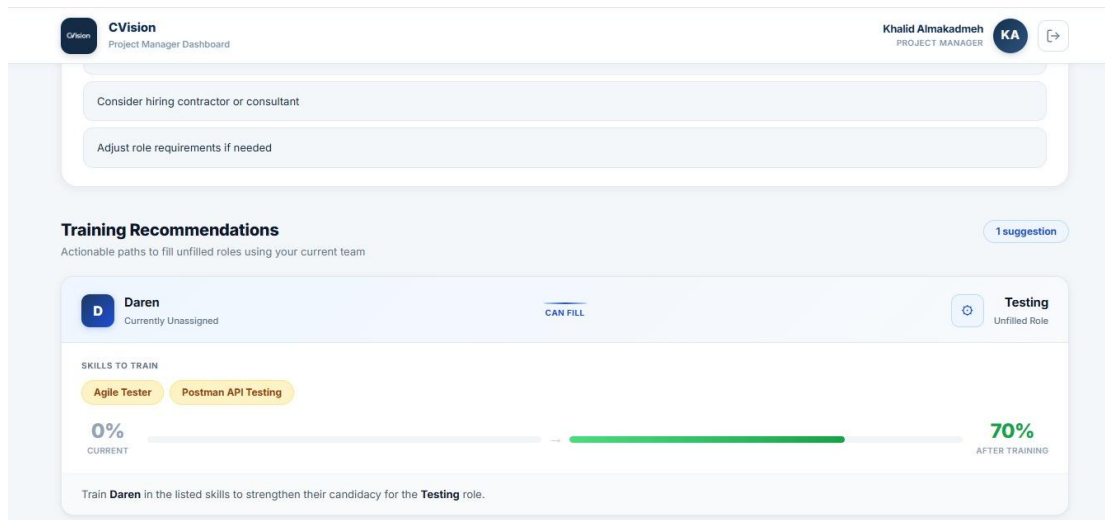


Figure 34: Training Recommendations

CHAPTER 5:IMPLEMENTATION PLAN

5.1 Description of Implementation

The CVision system is implemented as a web-based application that automates CV analysis, skill evaluation, role matching, and skill gap identification. The system consists of a React frontend, a FastAPI backend, a PostgreSQL database hosted on Neon, and Gemini AI for CV analysis.

Major Implementation Tasks

Backend Development

Implementation of backend services responsible for user authentication, project management, CV uploads, skill management, role matching, and API communication.

AI Integration

Integration of Gemini AI to analyze uploaded CVs, extract skills, certifications, tools, experience information, and generate proficiency scores.

Database Implementation

Implementation of the PostgreSQL database used to store users, projects, roles, skills, AI analysis results, role assignments, and role requirements.

Frontend Development

Development of a responsive web interface that enables users to upload CVs, manage projects, view dashboards, analyze skills, and review role matching results.

System Integration and Testing

Integration of all system components and validation of data flow between the frontend, backend, database, and AI service.

Resources Required

Hardware:

Developer laptops or personal computers with sufficient processing power for development and testing.

Software:

Python, FastAPI, React, PostgreSQL (Neon), Git, GitHub, Figma, and Gemini API.

Personnel:

Project team members responsible for backend development, frontend development, database management, testing, and documentation.

Facilities:

Internet connection and cloud-hosted services required for system operation.

5.2 Programming language and technology

The CVision system uses modern and widely adopted technologies to ensure reliability, maintainability, and scalability.

Backend Technologies

- **Python:** Used for backend development and business logic.
- **FastAPI:** Framework used to build RESTful APIs and manage server-side operations.

Frontend Technologies

- **JavaScript:** Core scripting language for frontend development.
- **React.js:** Used to build dynamic and responsive user interfaces.
- **HTML5 and CSS3:** Used for page structure and styling.

AI Technologies

- **Google Gemini API:** Used for CV analysis, skill extraction, skill evaluation, and role recommendation support.

Database and Storage

- **PostgreSQL (Neon):** Used to store users, projects, skills, role requirements, role assignments, and AI analysis results.
- **Local File Storage:** Used to store uploaded CV files.

Development and Design Tools

- **Figma:** Used to design and validate UI prototypes.
- **Draw.io:** Used for UML and architectural diagrams.
- **Git and GitHub:** Used for version control and collaboration.

These technologies were selected to provide a reliable, scalable, and maintainable solution for AI-powered CV analysis and role matching.

5.3 Version Control

Version control is an essential part of the CVision development process. All source code and project documentation are managed using **Git** and hosted on **GitHub** to support collaboration, track changes, and maintain project history.

A shared repository, hosted on **GitHub**, is used to:

- Track code changes and individual contributions from each team member.
- Maintain a clear and organized history of system evolution.
- Support collaboration and resolve code conflicts efficiently.

Each team member uses an individual GitHub account, and all commits follow clear and descriptive commit messages, such as:

- “Implemented CV upload API”
- “Added skill extraction module”
- “Improved dashboard visualization”

GitHub is also used to manage project versions and ensure that the latest stable code is available to all team members.

CHAPTER 6: TESTING PLAN

This chapter presents the testing plan for the CVision system. The purpose of testing is to verify that the system functions correctly and meets its functional and non-functional requirements before deployment.

1. Functional Testing

Functional testing verifies that the main features of the system operate as expected.

Examples in CVision:

- Verify user registration and login functionality.
- Verify CV upload using supported file formats (PDF and DOCX).
- Test AI-based CV analysis and skill extraction.
- Verify storage of extracted skills and proficiency scores.
- Validate individual skill dashboard generation.
- Verify project creation and member joining functionality.
- Test role matching based on role requirements and skill priorities.
- Verify skill gap analysis and missing skill identification.

2. Non-Functional Testing

Non-functional testing evaluates the quality attributes of the system, including performance, usability, reliability, security, and compatibility.

Examples in CVision:

- Measure CV analysis response time under normal usage conditions.
- Verify dashboard loading performance.
- Test system usability for both Project Managers and Team Members.
- Verify compatibility across modern web browsers.
- Validate secure authentication and password protection mechanisms.
- Verify proper handling of invalid inputs, upload errors, and AI service failures.

Testing Approach Summary

The testing plan for CVision focuses on validating the core system functionalities, including CV upload, AI-based CV analysis, skill extraction, role matching, and skill gap analysis. The approach combines black-box testing for user-facing features and white-box testing for backend logic and API functionality.

6.1 Black-box

Black-box testing validates system functionality from the user's perspective without considering the internal implementation. Test cases are designed based on system requirements and expected outputs.

Black-Box Testing Techniques

- **Equivalence Partitioning:**

Inputs are divided into valid and invalid categories. In CVision, this includes testing supported CV formats (PDF/DOCX) and unsupported file formats

- **Boundary Value Analysis:**

Testing focuses on values near system limits. Examples include testing minimum and maximum file sizes and validating required input fields.

- **Use Case–Based Testing:**

Test cases are derived from the system use cases. Examples include:

- User registration and login.
- Successful CV upload and analysis.
- Handling invalid CV uploads.
- Viewing extracted skills and skill scores.
- Generating role matching results.
- Identifying missing skills through skill gap analysis.

A test case template:

Test ID	Core Functionality	Description	Input	Expected Output
TC-01	CV Upload & Analysis	Upload CV with supported format	PDF or DOCX CV file	CV is accepted and sent for analysis
TC-02	CV Upload & Analysis	Upload CV with unsupported format	TXT file	Error message indicating invalid file format
TC-03	AI CV Analysis	Extract skills from uploaded CV	CV text content	Skills, tools, and experience are extracted
TC-04	Skill Storage	Store extracted skills correctly	Extracted skill set	Skills are saved and linked to the user
TC-05	Individual Dashboard	Display analyzed skills to team member	Member with processed CV	Skills and proficiency scores are displayed
TC-06	Team Dashboard	Aggregate team skills	Multiple members with CVs	Combined team skill overview is displayed
TC-07	Role Recommendation	Generate role recommendations	Project roles + team skills	Ranked candidates are generated per role
TC-08	Role Recommendation	Handle multiple candidates for same role	More than one qualified member	System identifies role conflicts
TC-09	Role Assignment	Apply manual override for role conflicts	Manager-selected candidate	Final role assignment is saved
TC-10	Skill Gap Detection	Identify missing or weak skills	Role requirements vs team skills	Missing skills or uncovered roles are displayed

Table 20: Test Case

The above test cases focus on the core functionalities that the CVision project is built upon, including CV analysis, skill visualization, and intelligent role recommendation. Supporting features such as authentication and account management are excluded to emphasize the system's primary objectives.

6.2 White-box

White-box testing focuses on verifying the internal logic of the CVision system. It is applied to backend functions such as user authentication, CV processing, AI analysis integration, role matching calculations, and skill gap analysis.

White-box Techniques

- **Statement Coverage:** Ensures that major code statements are executed during testing.
- **Branch Coverage:** Verifies both true and false outcomes of decision points within the system.
- **Path Testing (for critical functions)** Validates important execution paths such as successful analysis, invalid uploads, missing skills, and empty results.

6.3 Testing automation

Basic automated testing is used to verify backend APIs and core system functionality during development.

Automation Tools

- **Selenium (UI Automation):** automates end-to-end user flows such as CV upload, dashboard navigation, and generating recommendations through the web interface.
- **PyTest (Backend Automation):** automates unit tests for internal Python functions such as skill extraction, scoring, and candidate ranking.

Test Cases Selected for Automation

- User authentication and login validation.
- CV upload validation.
- AI analysis API testing.
- Role matching calculation testing.
- Skill gap analysis testing.

Automation Criteria

- Prioritize frequently executed tests.
- Focus on tests with predictable expected outputs.
- Re-run tests after major system updates to verify stability.

Manual testing remains important for validating user interface design, usability, and overall user experience.

CHAPTER 7: CONCLUSION AND RESULTS

This project addressed the challenge of assigning suitable roles within software development teams based on members' actual skills and qualifications. Traditional role assignment methods often rely on subjective judgments and limited information, which can lead to inefficient team utilization.

To address this problem, CVision was developed as an AI-powered web application that analyzes uploaded CVs, extracts technical and soft skills, evaluates skill proficiency, and supports role matching based on project requirements. The system provides a structured view of team capabilities and helps Project Managers make more informed decisions.

The implemented system successfully supports CV analysis, skill evaluation, role matching, skill gap identification, project management, and dashboard visualization. The results demonstrate that CVision can improve transparency, reduce manual effort, and support more effective team planning and role allocation.

7.1 Summary of accomplished project

- Developed a complete web-based system using React, FastAPI, PostgreSQL, and Gemini AI.
- Implemented user authentication and project management functionalities.
- Developed CV upload and AI-powered CV analysis features.
- Extracted technical skills, soft skills, tools, certifications, and experience information from uploaded CVs.
- Generated skill proficiency scores based on CV content.
- Implemented role matching using skill scores and role requirement priorities.
- Developed skill gap analysis functionality.
- Designed dashboards for visualizing user skills and project-related information.
- Tested and validated the system functionality using manual testing and API testing.

The completed system meets the main project objectives and provides a practical solution for skill-based role assignment in software development teams.

7.2 Future Work

Future enhancements may include:

- Supporting multilingual CV analysis.
- Adding advanced reporting and analytics features.
- Supporting larger teams and enterprise-scale projects.
- Enhancing dashboard visualizations and project management features.
- Integrating external professional platforms such as GitHub or LinkedIn.
- Improving recommendation accuracy using advanced machine learning models.
- Adding real-time analytics and extended reporting features.

These future enhancements would increase the scalability, intelligence, and practical value of the CVision system.

REFERENCES

- [1] Figma, “Collaborative Interface Design Tool,” 2024. [Online].
Available: <https://www.figma.com/>.
- [2] Draw.io, “Online Diagramming and Flowchart Tool,” 2024. [Online].
Available: <https://www.drawio.com/>.
- [3] Discord, “Communication and Collaboration Platform,” 2024. [Online].
Available: <https://discord.com/>.
- [4] Canva, “Online Graphic Design Platform,” 2024. [Online].
Available: <https://www.canva.com/>.
- [5] Pinterest, “Visual Inspiration Platform,” 2024. [Online].
Available: <https://www.pinterest.com/>.
- [6] Google Search, “Software Architecture and UML Diagrams,” 2024. [Online].
Available: <https://www.google.com/>.
- [7] OpenAI, “ChatGPT: AI-Based Text Generation and Writing Assistance Tool,” 2025. [Online].
Available: <https://chat.openai.com/>.
- [8] FastAPI, “FastAPI Framework Documentation,” 2025. [Online].
Available: <https://fastapi.tiangolo.com/>
- [9] React, “React Documentation,” 2025. [Online].
Available: <https://react.dev/>
- [10] PostgreSQL, “PostgreSQL Documentation,” 2025. [Online].
Available: <https://www.postgresql.org/docs/>
- [11] Neon, “Serverless PostgreSQL Platform,” 2025. [Online].
Available: <https://neon.tech/>

[12] Google AI, “Gemini API Documentation,” 2025. [Online].

Available: <https://ai.google.dev/>

[13] Anthropic, “Claude AI Assistant,” 2025. [Online].

Available: <https://claude.ai/>